



GRADO EN INGENIERÍA
MULTIMEDIA



VNIVERSITAT
DE VALÈNCIA

TRABAJO FIN DE GRADO

DISEÑO E IMPLEMENTACIÓN DE UN TECLADO
VIRTUAL MULTILINGÜE PARA PROYECTOS DE VR

AUTOR: JONATHAN DAVID SIGNES FALCÓ

TUTOR: JAVIER PERIS SEVILLA

MARZO 2025



VNIVERSITAT
DE VALÈNCIA



Escola Tècnica Superior
d'Enginyeria **ETSE-UV**

TRABAJO FIN DE GRADO

DISEÑO E IMPLEMENTACIÓN DE UN TECLADO
VIRTUAL MULTILINGÜE PARA PROYECTOS DE VR

AUTOR: JONATHAN DAVID SIGNES FALCÓ

TUTOR: JAVIER PERIS SEVILLA

Declaración de autoría:

Yo, Jonathan David Signes Falcó, declaro la autoría del Trabajo Fin de Grado titulado “Diseño e implementación de un teclado virtual multilingüe para proyectos de VR” y que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual. El material no original que figura en este trabajo ha sido atribuido a sus legítimos autores.

Valencia, 23 de junio de 2025

Fdo: Jonathan David Signes Falcó

Resumen:

Diseño e implementación de un teclado virtual configurable para tener una simulación de un teclado físico y un teclado plano.

Está hecho con Unity e implementado en siete idiomas cuyos teclados usan sus propios caracteres. Entre estos están el ruso, inglés, alemán, francés, español, griego y coreano.

Abstract:

Design and implementation of a configurable virtual keyboard to simulate a physical keyboard and a flat keyboard.

It is built with Unity and implemented in seven languages, whose keyboards use their own characters. These include Russian, English, German, French, Spanish, Greek, and Korean.

Resum:

Design and implementation of a configurable virtual keyboard to simulate a physical keyboard and a flat keyboard.

This would be made with Unity and would implement seven languages whose keyboards use their own characters. These would include Russian, English, German, French, Spanish, Greek and Korean.

Agradecimientos:

En primer lugar, quiero agradecer a todos aquellos que me han apoyado durante estos años, en especial al profesorado que me ha acompañado en las distintas etapas de mi vida académica, han sabido ver en mí, valores y capacidades que han ido desarrollándose.

Asimismo, quiero dar las gracias a mi familia por apoyarme incondicionalmente y soportar mis desvelos con cariño y paciencia. También he de agradecer a mi psicóloga el hecho de ayudarme a aclarar mi mente cuando me daban bajones emocionales que me impedían continuar.

Por último y no menos importante, a las inteligencias artificiales al proporcionarme una gran cantidad de estudios, artículos y páginas web para profundizar mi conocimiento en lo que respecta a la tecnología VR y su interacción con las personas e ideas cuando me he quedado atorado en algún punto porque no se me ocurría qué más poner.

Índice general

1. Introducción	19
1.1. Introducción	19
1.2. Motivación	20
1.3. Objetivos	20
1.4. Organización de la memoria	20
2. Estado del arte	23
2.1. Análisis de aplicaciones similares	23
2.2. Tecnologías	23
2.2.1. Tecnologías de Desarrollo	23
2.2.2. Sistemas de Despliegue	24
2.3. Evolución de los teclados en realidad virtual	24
2.3.1. Orígenes	24
2.3.2. Las dificultades identificadas en estos primeros modelos	25
2.3.3. Nuevas sugerencias para incorporar texto en Realidad Virtual	25
2.4. Necesidad de soporte multilingüe en VR	26
2.4.1. Accesibilidad lingüística en entornos digitales	26
2.4.2. Dificultad para representar escrituras no latinas en VR	27
2.4.3. Cómo otros campos han incorporado el multilinguaje y el rezago de la VR en este aspecto	27
2.4.4. Globalización de usuarios de VR y pérdida de un mercado potencial	28
2.5. Interfaces naturales de usuario (NUI) en VR	28
2.5.1. Definición y contexto de las NUIs	28
2.5.2. Tecnologías principales en las NUIs para VR	29
2.5.3. Implementación de NUI en este proyecto	30
2.5.4. Beneficios de usar NUI en teclados virtuales	30
2.6. Retos de accesibilidad en realidad virtual	31
2.6.1. Barreras actuales en VR para usuarios diversos	31
2.6.2. Necesidad de accesibilidad en teclados VR	32

2.6.3.	Cómo este proyecto aborda la accesibilidad	33
2.6.4.	Limitaciones actuales y posibles mejoras	34
2.7.	Estándares y tendencias de mercado en VR	34
2.7.1.	Estado actual del mercado de VR	34
2.7.2.	Estándares emergentes en VR	35
2.8.	Selección y justificación de idiomas implementados	36
2.8.1.	Criterios de selección	36
2.8.2.	Idiomas implementados	36
2.8.3.	Idiomas descartados y motivos	37
2.8.4.	Retos técnicos en la implementación	38
3.	Requisitos, especificaciones, coste, riesgos, viabilidad	39
3.1.	Requisitos	39
3.2.	Especificaciones	39
3.3.	Especificaciones	39
3.4.	Costes	41
3.5.	Riesgos	42
3.6.	Viabilidad	44
3.6.1.	Viabilidad técnica	44
3.6.2.	Viabilidad económica	45
3.6.3.	Viabilidad temporal	45
3.6.4.	Conclusión sobre la viabilidad	46
4.	Análisis	47
4.1.	Introducción	47
4.2.	Diagrama de casos de uso	47
4.2.1.	Descripción	48
4.3.	Modelo del dominio	48
4.3.1.	Descripción del diagrama	49
4.3.2.	Principios SOLID que se siguen	50
4.4.	Análisis de comportamiento	51
4.4.1.	Diagramas de actividad	52
5.	Diseño	55
5.1.	Introducción	55
5.1.1.	Objetivos	55
5.1.2.	Relación con el análisis	55
5.2.	Diseño de la Arquitectura	56

5.2.1.	Arquitectura del sistema	56
5.2.2.	Diagrama de despliegue	57
5.3.	Diagrama de clases definitivo	57
5.3.1.	Descripción del diagrama de clases	58
5.3.2.	Apéndices y aclaraciones	61
5.4.	Diagramas de secuencia	61
5.4.1.	Pulsado de tecla	61
5.4.2.	Cambiar de idioma	62
5.4.3.	Cambiar de tipo de teclado	62
5.4.4.	Modificar letras	63
5.4.5.	KeyboardWriter	65
5.4.6.	Insertar carácter	67
5.4.7.	Otras funciones del keyboard	69
5.4.8.	Inicio del programa	70
6.	Implementación y pruebas	73
6.1.	Implementación	73
6.1.1.	Tecnologías y herramientas	73
6.1.2.	Desafíos técnicos específicos	73
6.2.	Validando el sistema: Mi experiencia práctica	75
6.3.	Pruebas de rendimiento	76
6.4.	Lo que aprendí en el camino	76
6.5.	Pruebas de usabilidad	76
6.5.1.	Metodología SUS	76
6.5.2.	Video de demostración	79
7.	Conclusiones	81
7.1.	Revisión de costes	81
7.1.1.	Tiempo: Mi recurso más gastado	81
7.1.2.	La batalla de los idiomas	81
7.1.3.	Recursos: Mi equipo sufriendo conmigo	81
7.2.	Conclusiones	82
7.2.1.	Triunfos técnicos	82
7.2.2.	Sorpresas agradables	82
7.2.3.	Lecciones aprendidas	82
7.3.	Trabajo futuro	82
A.	Apéndice	85

A.1. Apéndice del capítulo 3.2	85
A.2. Apéndice del capítulo 5.3	85
A.3. Apéndice del capítulo 6	87
Bibliografía	88

Capítulo 1

Introducción

1.1. Introducción

El avance de la realidad virtual (VR) está creciendo rápido, y con él, la necesidad de que los desarrolladores puedan crear experiencias inmersivas que se adapten a todo tipo de usuarios. No todos interactúan igual con la tecnología, así que hacen falta herramientas flexibles, intuitivas y, sobre todo, que entiendan las diferencias entre las personas. Uno de los grandes inconvenientes de la realidad virtual en la actualidad es que parece estar exclusivamente concebida para el idioma inglés. ¿Has buscado un teclado en Realidad Virtual que te permita redactar en español, francés o árabe? No lo hay. Visualiza que tu lengua materna, la que te vincula con tus orígenes y tu cultura, sea considerada un fallo de sistema: es la situación que millones de personas experimentan cuando la tecnología es concebida desde una burbuja anglocéntrica, les demanda ajustarse o callar. Investigaciones como la de [1] no solo descubren que las voces con acentos o abecedarios no ingleses son interpretadas como “ruido”, sino que nos exponen a una realidad dura: la marginación lingüística en la Realidad Virtual y otras tecnologías no es un fallo, sino una herida colonial que arde en plena era digital. Hablamos de abuelas que no pueden dictar mensajes, de niños que ven su escritura autocorregida hasta volverse irreconocible, de lenguas enteras relegadas al estatus de “exóticas” en algoritmos que, por diseño, las silencian. La solución no está en remiendos técnicos, sino en desaprender la idea de que lo universal debe sonar a inglés.

Es cierto que hay lenguas, como las romances, cuyo único problema es la falta de algunos caracteres, que a las malas puedes añadir tu fácilmente en tu teclado, pero este problema es mucho mayor en regiones como el Sudeste Asiático o en grupos de hablantes que utilizan tipos de escritura distintos al alfabeto latino. Situaciones tales como el Hangul coreano o la escritura árabe, con trazos asociados y orientaciones concretas, demuestran la importancia de rediseñar las interfaces de entrada en entornos virtuales [2].

Como solución a estos retos, varias investigaciones han explorado alternativas como la modificación del comportamiento de los teclados físicos en el espacio virtual. Estas tácticas facilitan la modificación dinámica de la configuración y funcionamiento de las teclas, generando nuevas oportunidades para la introducción de textos multilingües sin la necesidad de modificar el hardware [3]. En resumen, estas sugerencias subrayan la necesidad apremiante de crear soluciones de diseño que tengan en cuenta la variedad lingüística y cultural de los usuarios en experiencias envolventes.

1.2. Motivación

La idea de este Asset me surgió durante la realización de las prácticas curriculares en el Instituto de Robótica y Tecnologías de la Información y la Comunicación (IRTIC) donde se me pidió un teclado virtual para uno de los proyectos que se estaban realizando con Unity.

Al buscar en el Asset Store (tienda oficial de Unity), no encontré nada que se adaptara tanto a las necesidades idiomáticas como presupuestarias. Por lo que el equipo decidió hacer un teclado a mano muy básico.

Tras la realización de este, uno de los tutores de la empresa, que también resultaba ser profesor de la universidad, me sugirió la idea de hacerlo mi TFG, puesto que, para entonces aun no tenía una idea clara de qué elegir como producto de mi trabajo, pero añadiéndole más funcionalidades, como diseñarlo para varios idiomas posibilitando la captación de otros mercados que también sufren el hecho de tener que recurrir a la lengua anglosajona.

1.3. Objetivos

Este proyecto se enfoca en el desarrollo y construcción de un Asset de un teclado multilingüe para ambientes de realidad virtual en la plataforma Unity. Se deben tener en cuenta tres principios fundamentales:

1. La satisfacción de necesidades específicas detectadas durante las prácticas en el Instituto de Robótica y Tecnologías de la Información y la Comunicación (IRTIC):
 - La ausencia de teclados españoles para Realidad Virtual y Realidad Virtual.
2. La oferta de una reacción competitiva mediante:
 - Soporte intuitivo en siete idiomas (español, inglés, francés, alemán, griego, ruso y coreano)
 - Cambio rápido entre las distribuciones sin reiniciar la escena.
3. Garantías de habilidad para adaptarse a:
 - cualquier proyecto en Unity que emplee la Realidad Virtual.
 - Diversos métodos de entrada (mandos VR, Hand Tracking)

1.4. Organización de la memoria

La memoria se organizará de la siguiente manera:

- **Estado del Arte (Capítulo 2):** en este punto se realizará una revisión de las tecnologías utilizadas para la realización del teclado, de aplicaciones parecidas y un estudio teórico de todos los puntos a los que se puede relacionar este proyecto, utilizando diversos artículos y estudios.

- **Requisitos, especificaciones, coste, riesgos, viabilidad (Capítulo 3):** en este apartado se abordan los aspectos clave del proyecto relacionados con la planificación, la fundamentación, detalles técnicos de la implementación, los costes para llevar a cabo éste, identificación y planes de acción de riesgos que puedan surgir y un análisis técnico, económico y operativo para comprender si se puede llevar a cabo el proyecto de una manera sostenible.
- **Análisis (Capítulo 4):** en esta parte del TFG se realiza una revisión exhaustiva de las tecnologías utilizadas para la realización del teclado.
- **Diseño (Capítulo 5):** la realización de una descripción detallada de los distintos diseños utilizados como la interfaz de usuarios, algoritmos utilizados y funcionalidades específicas es el centro de estudio de este capítulo.
- **Implementación y pruebas (Capítulo 6):** este punto del trabajo comporta la realización de una revisión exhaustiva de las tecnologías utilizadas para la realización del teclado y de las distintas pruebas que se llevarán a cabo.
- **Conclusiones (Capítulo 7):** se resumen los logros alcanzados, las implicaciones prácticas y teóricas del teclado diseñado, y se proponen direcciones para futuras actualizaciones.
- **Apéndice :** Aquí se adjunta material adicional, como el código fuente, datos adicionales y gráficos detallados que respaldan el contenido principal de la memoria.
- **Bibliografía:** incluye una lista completa de todas las fuentes citadas en el trabajo.

Capítulo 2

Estado del arte

2.1. Análisis de aplicaciones similares

Como el TFG no se trata de una **aplicación per se**, sino de un **asset** destinado al **Asset Store de Unity**, se puede determinar que **no hay aplicaciones similares** en el mercado tecnológico. No obstante, sí se pueden encontrar **veintinueve Assets de teclados para realidad virtual** en total, que oscilan entre la **gratuidad** y precios de hasta **359.27 €**.

Los **teclados de gama baja** parecen muy simples respecto a **uso, funciones y customización**. Los **de gama media** son muy **personalizables**. En cuanto a la **gama alta de teclados**, destaca su **robustez** y capacidad para **usos múltiples en todo el sistema**.

Para **distinguirme** de todos estos, decidí diseñar un teclado con el que se pudiese escribir en **varios idiomas**, ya que ninguno de los veintinueve teclados antes mencionados poseen esta característica, acortando así su rango de mercado, puesto que se limitan a la **lengua inglesa**.

2.2. Tecnologías

Para el desarrollo del **teclado multilingüe en realidad virtual**, se han considerado diversas **tecnologías y sistemas de despliegue**. A continuación, se presentan las **opciones analizadas** y las **razones detrás de la selección final**.

2.2.1. Tecnologías de Desarrollo

- **Unity**: fue seleccionado como la **principal plataforma de desarrollo** por su **flexibilidad y robustez de soporte** para aplicaciones de VR. Su capacidad de integración con **pluguins como TextMeshPro** y mi **estudio y experiencia con ella**, gracias a mi formación universitaria, fueron factores decisivos. Hay que añadir que permite el desarrollo en **diversas plataformas**, con lo que se alcanza una **mayor audiencia**, sumándose al factor de los **múltiples idiomas**.

- **Unreal Engine**: Aunque Unreal Engine ofrece **gráficos de alta calidad** y herramientas **avanzadas** para el desarrollo de VR; su **curva de aprendizaje es más**

pronunciada y, dado que debería haber aprendido de cero, el tiempo requerido para la realización del proyecto se habría **prolongado excesivamente**, volviéndose **inviable**, para terminar en el plazo requerido. Dado el enfoque del proyecto en la **usabilidad** y la **eficiencia del desarrollo**, Unity fue la **opción preferida**. Además, la **gestión de recursos** y la **documentación extensa** en Unity facilitaron la implementación del **teclado multilingüe**.

2.2.2. Sistemas de Despliegue

- **Asset Store**: fue seleccionado como uno de los **sistemas de despliegue** por distintos motivos. Es una **plataforma muy conocida** y **ampliamente utilizada** por los desarrolladores, lo que brinda una **altísima exposición**. El hecho de que el asset esté disponible en el Asset Store de Unity puede servir como **validación de su calidad y utilidad**. Así pues, los usuarios pueden confiar en que estos activos han pasado por distintos **procesos de revisión** y cumpliendo, de esta manera, con los **estándares de calidad de Unity**, y aumentando la **credibilidad** y **confianza en el producto**. Es muy fácil **descubrir**, **adquirir** y **administrar** los distintos assets, puesto que emplea la **infraestructura existente de Unity** para su distribución y gestión. La publicación aquí permite **actualizar regularmente los assets** y **ofrecer soporte a los usuarios de manera eficiente**. Pueden recibir **notificaciones automáticas de actualizaciones** y acceder fácilmente a la **documentación** y el **soporte técnico** que proporciones a través de la plataforma. Y, por último, pero no menos importante, ofrece la posibilidad de **monetizar tu trabajo** vendiendo tu asset a otros desarrolladores, aspecto que es parte del **objetivo de este proyecto**.

- **GitHub**: Aunque GitHub también es considerablemente conocido por los desarrolladores, la integración con **GitHub Actions** para la **automatización de flujos de trabajo de CI/CD** ayuda muchísimo a la hora de **ejecutar pruebas** y el **control de versiones**; no obstante, el hecho de que tuviese que **aprender de cero** el funcionamiento de **GitHub Actions** y mi **objetivo de comercializar el teclado**, me hacen poco atractiva esta idea. Dicho esto, sigo utilizándolo para **guardar el proyecto para mi uso personal** y no rechazo la idea de **utilizarlo para publicitarme como programador**.

2.3. Evolución de los teclados en realidad virtual

2.3.1. Orígenes

A mediados de 2010, los sistemas de realidad virtual (VR) comenzaron a ganar importancia, principalmente con Oculus Rift, HTC Vive y, posteriormente, con Quest, uno de los desafíos más significativos era la posibilidad de que el usuario pudiese añadir texto a estos universos virtuales.

Inicialmente, la solución más clara fue modificar los teclados físicos tradicionales que utilizamos en ordenadores de escritorio:

- Se sugería que el usuario simplemente tecleara en su teclado físico real, mientras veía una representación virtual de sus manos o del propio teclado.

- Incluso algunos sistemas iniciales de Realidad Virtual no mostraban contenido: confiaban en que el usuario entrara “de memoria”.
- Otras visiones empleaban cámaras de ubicación externa o sensores de movimiento de saltos para capturar los dedos en tiempo real y proyectarlos en el espacio virtual.

2.3.2. Las dificultades identificadas en estos primeros modelos

La puesta en práctica de este enfoque pronto dejó al descubierto serios problemas que lastraban su utilidad real. Para empezar, los errores al teclear se multiplicaban porque los usuarios, al no tener visión directa del teclado, perdían constantemente la orientación sobre dónde estaban las teclas. Pero esto era solo el principio: según documenta [2] y otros trabajos, escribir en estos entornos virtuales resultaba notablemente más lento —entre un 30 % y hasta un 60 %— que usar un teclado físico común.

La cosa se complicaba con la ergonomía. Resulta que mantener los brazos estirados para alcanzar el teclado mientras cargabas con ese visor enorme forzaba posturas poco naturales, algo que los usuarios reportaban como bastante incómodo tras poco tiempo de uso. A esto se sumaba un problema técnico persistente: con frecuencia, lo que veían en el visor no coincidía bien con la posición real del teclado; ese desfase visual —lo llamaban “desincronización”— hacía que teclear fuese como intentar encestar con los ojos cerrados.

Y había un último escollo crucial: el enfoque dependía por completo de periféricos externos. Esto chocaba frontalmente con la esencia de dispositivos como el Oculus Quest, que precisamente buscan liberarte de cables y accesorios. En la práctica, esa dependencia los hacía poco viables para escenarios móviles donde lo que prima es la autonomía.

2.3.3. Nuevas sugerencias para incorporar texto en Realidad Virtual

Para resolver estos desafíos, empezaron a aparecer **innovaciones concretas de realidad virtual** en el campo de la introducción de texto:

Teclados virtuales flotantes

- Se comenzaron a diseñar **teclados completamente virtuales** que flotaban frente al usuario en el espacio VR.
- Estos teclados tenían la capacidad de **modificar su tamaño, ubicación y curvatura** para ajustarse de manera más efectiva al ángulo natural de los brazos.
- Algunos teclados facilitan la escritura **al apuntar** con los controladores, al pulsar con los dedos o incluso al hacer gestos con la mano.

Predicción de texto y autocorrección

Con la intención de aumentar la velocidad de escritura y minimizar los errores ortográficos, se implantaron sistemas de **predicción de palabras**, similares a los utilizados en los smartphones. Esto permite que el usuario no necesite escribir una palabra completa,

acelerando, de esta manera, el proceso de escritura notablemente y, a su vez, mejorando la experiencia. Los ejemplos presentes incluyen predictores de contexto en teclados de aplicaciones sociales de Realidad Virtual (como VRChat y Horizon Worlds).

Combinación con comandos de voz

Una nueva implementación que se realizó fue el **reconocimiento de voz**, que tenía el objetivo de reducir el tecleo en la virtualidad. Esta novedad posibilitó que el usuario fuese capaz de crear parte del texto hablando y, posteriormente, corregir o cambiar pequeños fragmentos de su producción escrita a través de un teclado virtual. Sin embargo, los sistemas de voz tenían, y siguen teniendo, limitaciones: ruido del entorno, falta de privacidad, dificultades con nombres propios o con vocablos extranjeros.

Resultados de estudios recientes

En investigaciones más actuales, como las realizadas por Rojas y colaboradores (2024) [1] y Peng y colaboradores (2024) [2], se ha corroborado lo siguiente:

- Los usuarios eligen **soluciones híbridas** (una combinación tanto de voz, como predicción de texto y, por supuesto, teclados adaptables).
- Los teclados físicos siguen destacando por su velocidad, mientras que los teclados virtuales adecuadamente diseñados pueden superarlos ampliamente en eficacia.
- Hay una enorme falta de soluciones multilingües: la mayoría de los teclados virtuales actuales están diseñados únicamente para el inglés, lo que restringe su adopción a nivel internacional.

2.4. Necesidad de soporte multilingüe en VR

2.4.1. Accesibilidad lingüística en entornos digitales

Es fundamental la accesibilidad lingüística en entornos digitales para garantizar la inclusión y la equidad en el uso de la tecnología. En el contexto de la realidad virtual (VR), se intensifica gradualmente la relevancia de incorporar diferentes lenguas en interfaces como los teclados virtuales, ya que la realidad virtual se utiliza en sectores mundiales como la educación, el trabajo y el ocio. Si los usuarios no logran interactuar en su propio idioma, la experiencia se vuelve limitada y se corre el riesgo de excluir a grandes grupos de la población mundial, lo que podría llevar a una disminución del uso y a la pérdida de oportunidades de mercado. [4] [5] [6]

Investigaciones actuales han evidenciado que la incorporación de Realidad Virtual en la orientación y el aprendizaje en varios idiomas puede funcionar como una conexión entre el idioma nativo de los alumnos y los temas educativos. Por ejemplo, en el sector de la educación, la guía multilingüe en Realidad Virtual ha permitido a los estudiantes acceder a contenidos en su lengua materna, mejorando así la comprensión conceptual y la calidad de su aprendizaje. Los profesores actúan como pilares de apoyo, utilizando ambientes de Realidad Virtual flexibles para generar experiencias educativas que cumplan con las

necesidades comunicativas de los estudiantes, promoviendo, así, situaciones de aprendizaje caracterizadas por la equidad e inclusión en las aulas. [4] [5]

Los estudios al respecto señalan que los ambientes virtuales fomentan la adquisición de contenidos y competencias, así como la consecución del desarrollo integral del alumnado. Los estudiantes multilingües suelen lidiar con problemas extra en diversas ramas, como las científicas. Las técnicas convencionales, enfocadas en el lenguaje verbal, tienen restricciones que la Realidad Virtual contribuye a superar, ya que cuenta con la capacidad de generar significado multimodal (visual, gestual, verbal); esto favorece todos los tipos de alumnos. [6]

No se debe restringir la relevancia de la accesibilidad lingüística, en este contexto de Realidad Virtual, únicamente a la inclusión en la educación. Esta es capaz de establecer comunidades mundiales de aprendizaje y cooperación sin obstáculos lingüísticos para una participación activa y relevante. [4] [5] [6]

2.4.2. Dificultad para representar escrituras no latinas en VR

La representación de las lenguas no románicas, tales como árabe, japonés, chino, entre otros, es uno de los retos técnicos más complicados al implementar soporte multilingüe, no solo en RV. Estos sistemas de escritura presentan desafíos únicos en cuanto a visualización y reconocimiento de caracteres y, al intentar ofrecer experiencias auténticas y envolventes, el desafío no hace más que aumentar. Una de estas especificaciones es la personalización de las teclas. [7] [8]

La versatilidad de la Realidad Virtual facilita la búsqueda de nuevas modalidades de interacción y visualización para respaldar estos sistemas de escritura. No obstante, la mayoría de las soluciones actuales todavía se enfocan en lenguas occidentales y no tratan de manera integral la variedad de escritura mundial.

Frente a la ausencia de soluciones adecuadas para la representación de lenguas ajenas a la familia románica, millones de personas se ven coaccionadas a no poder disfrutar experiencias en Realidad Virtual. Esto, lo único que hace es perpetuar la desigualdad en el acceso a la tecnología y la información. Para llegar a la meta de una Realidad Virtual realmente global e inclusiva, es necesario vencer estos retos. [7] [8]

2.4.3. Cómo otros campos han incorporado el multilinguaje y el rezago de la VR en este aspecto

En los videojuegos y las aplicaciones para smartphones, entre otros campos, se considera común la compatibilidad en múltiples lenguas. Estas plataformas generalmente disponen de interfaces que habilitan a los usuarios a elegir un idioma concreto y a utilizar diseños ajustados a distintos alfabetos y sistemas de escritura. Esto ha propiciado la adopción global de los diferentes productos y ha optimizado la experiencia del usuario. [9] [10]

Dicho esto, hay que confesar que desgraciadamente la Realidad Virtual no ha visto grandes avances en esta área. La mayoría de los teclados virtuales en Realidad Virtual continúan empleando diseños estándar y sin alternativas multilingües decentes. A pesar de que hay herramientas y kits de desarrollo que posibilitan una cierta personalización, la incorporación de soporte para varios idiomas y escrituras aún es restringida y, en nume-

rosas situaciones, demanda soluciones personalizadas por parte de los programadores[4] [8] [11]. Esta inequidad obstaculiza la aplicación de la Realidad Virtual en mercados que no sean ingleses o que empleen sistemas de escritura distintos al de las lenguas romances.

La posibilidad de modificar teclados tanto físicos como virtuales en la Realidad Virtual, tal como se analiza en ReconViguRation, genera nuevas posibilidades para la personalización y el respaldo en diferentes idiomas. No obstante, la implementación de estas soluciones todavía es reciente y necesita más análisis y progreso para llegar al grado de madurez que se observa en otras áreas digitales. [7] [8]

2.4.4. Globalización de usuarios de VR y pérdida de un mercado potencial

En las últimas décadas, la globalización ha cambiado el perfil de los usuarios de VR, como muchos otros perfiles, dado que, actualmente, provienen de una amplia variedad de países culturas y lenguas. La repercusión social y las posibilidades de mercado están restringidas debido a la marginación de una gran parte de la población, además de la falta de soporte en Realidad Virtual para varios idiomas. La dificultad para acceder al idioma puede llevar a una disminución en la utilización, una reducción en la satisfacción del usuario y, finalmente, a la pérdida de oportunidades en el sector comercial y educativo.. [4] [5] [9] [6] [10]

Como ya se ha visto anteriormente, la creación de ambientes de Realidad Virtual multilingües no solo promueve la inclusión y la equidad, sino que también facilita la formación de comunidades mundiales de aprendizaje, cooperación y diversión. Compañías y programadores que opten por la accesibilidad lingüística tendrán la posibilidad de ingresar en mercados en desarrollo variados, potenciando así su competitividad e importancia en un mundo cada vez más interrelacionado. Así mismo, la oportunidad de comunicarse en la lengua materna fomenta la participación activa y el sentimiento de pertenencia de los usuarios, elementos esenciales para el triunfo y la perdurabilidad de las plataformas de Realidad Virtual. [4] [5] [9] [6] [10]

2.5. Interfaces naturales de usuario (NUI) en VR

2.5.1. Definición y contexto de las NUIs

Este tipo de interfaces permiten a los usuarios interactuar con sistemas informáticos de manera más sencilla, directa y natural. Para lograr este objetivo, se utilizan comportamientos humanos tan comunes como la voz, el tacto, los gestos o la mirada, que, a pesar de no ser comunes para la máquina, mejoran notablemente la experiencia del usuario. Éstas interfaces tienen como objetivo disminuir la curva de aprendizaje, mejorar la accesibilidad y prevenir el uso de hardware excesivo como el teclado o el ratón. Éstas surgieron a raíz de la necesidad que éstas fueran más fáciles de aprender, debido, sobre todo, a la poca experiencia de algunos usuarios en el uso de la tecnología o de otros usuarios con necesidades especiales. Al basarse en comportamientos “naturales” e imitar las interacciones sensoriomotoras humanas, se elimina así la necesidad de aprender nuevas “mecánicas” recortando la curva de aprendizaje en el camino. [12] [13] [14] [15]

Ejemplos de NUIs incluyen asistentes de voz como Alexa o Siri, sistemas de control por

gestos en videojuegos, y dispositivos móviles con pantallas táctiles, entre otros muchos. [16]

Las interfaces de usuario tradicionales, como botones, menús, teclado y ratón, entre otros, se basan en la gestión de componentes visuales perceptibles y demandan el aprendizaje o desarrollo de nuevas competencias para interactuar con el sistema. Éstas se caracterizan por su visibilidad y exploración, permitiendo que los usuarios descubran las opciones disponibles a través de menús e ilustraciones, hecho que contrasta mucho con el enfoque de las NUIs, que se ha visto. No obstante, a pesar de que las NUIs pueden ser consideradas más naturales y envolventes, aún se enfrentan a desafíos en cuanto a usabilidad y retroalimentación, por lo que no siempre sobrepasan en eficiencia a las interfaces convencionales en todas las actividades, con lo cual, se puede considerar que queda un largo camino que recorrer en este sentido. [12] [17] [16]

2.5.2. Tecnologías principales en las NUIs para VR

Hand Tracking

A ver, si hay una tecnología que realmente cambia las reglas del juego en las NUIs de RV, es esta. Sin pelos en la lengua: el hand tracking es probablemente la más revolucionaria de todas. Básicamente, te permite trastear con objetos virtuales usando solo tus gestos, quitando de en medio esos engorrosos mandos.

Pongamos el caso del sistema NUI-VR², que es un buen ejemplo de lo que hablo. Este sistema —según hemos podido verificar— integra la manipulación de volúmenes 3D en RV de tal modo que tus manos se convierten en la herramienta principal para "dialogar" con los datos. La cosa no es solo más intuitiva; es que los usuarios acaban desarrollando una comprensión espacial mucho más sólida, casi sin darse cuenta.

Y, por contra, mira métodos como Zoom-fwd: ahí se ve claramente que un buen reconocimiento de gestos puede dar un vuelco a la precisión cuando hay que seleccionar objetos escondidos o lejanos. Vamos, que la cosa funciona [18] [19] [20].

Para rematar, en este Asset nos hemos tirado a la piscina con el hand-tracking usando el kit del Meta Quest 2 (antes Oculus Quest II, claro, después del cambio de nombre que hizo la empresa en octubre del 2021) [21].

Voice Recognition

El reconocimiento de voz, tecnología de bastante relevancia en el campo actual de las NUIs para VR, no siempre se utiliza, como es mi caso en este proyecto. Permite controlar hechos y emitir órdenes por la voz, lo que puede complementar o incluso sustituir otras formas de interacción, especialmente en situaciones en que las manos están ocupadas, y no pueden ser detectadas con precisión o no se puedan utilizar por motivos varios. Esta tecnología se suele combinar con otras para ampliar las posibilidades de interacción natural y accesibilidad. Aunque no todos los sistemas de VR implementan esta tecnología, su potencial para mejorar la usabilidad y la accesibilidad es ampliamente reconocido. [22] [23]

Eye Tracking

Esta tecnología ya es más avanzada y difícil de ver en la vida diaria. Se basa en detectar la dirección de la mirada del usuario y responder acorde a esa direccionalidad. Estas interacciones son muy rápidas en comparación a otras. Se suele usar para complementar otras tecnologías, como las ya mencionadas, para conseguir experiencias aún más inmersivas y eficientes. El seguimiento ocular se puede utilizar con muchos enfoques, pasando por la selección de objetos, navegar por menús y ajustar el enfoque de las escenas, cosa que viene muy bien a personas con problemas leves de vista, como yo mismo he experimentado, puesto que siento que veo mejor sin mis gafas de vista cuando uso los cascos. [23]

Body Tracking

El body tracking, como el nombre indica, se trata de la detección de posturas y gestos hechos con todo el cuerpo. Esta tecnología se usa en aplicaciones donde los movimientos corporales amplios o las posturas específicas son indispensables para la interacción, como en simulaciones, entrenamiento físico o experiencias de entretenimiento. En el VR, combinado con el hand tracking, elimina gran cantidad de dispositivos y permite mucha libertad de movimiento y desentraña un sinfín de posibilidades de evolución. [22] [20]

En cambio, se ha comprobado que la incorporación de sensores de movimiento y tecnologías de reconocimiento facial o corporal es eficaz para diseñar interfaces más intuitivas y fácilmente accesibles, incluso en situaciones donde el usuario emplee un head-mounted display (HMD). [24] [25]

2.5.3. Implementación de NUI en este proyecto

Como ya se ha vaticinado, en este proyecto he hecho uso de NUIs. He preparado el Asset del teclado para poder interactuar con el hand tracking y así hacerlo más cómodo para personas ajenas a la Realidad Virtual, pero que al menos sepan usar un teclado, como es el caso de mi madre o de mi tía abuela (mecnógrafa de profesión). Como ya he mencionado, la prueba de su funcionamiento se ha realizado con unas gafas meta Quest 2, por lo que no he podido corroborar el funcionamiento en más tipos de dispositivos, pero en un principio se ha estandarizado al emplear el “VR Interaction Toolkit” de Unity, herramienta que ofrece soporte a la mayoría de HUDs del mercado.

2.5.4. Beneficios de usar NUI en teclados virtuales

Su uso ofrece ventajas muy significativas y no refutables en el apartado de inmersión, accesibilidad y realismo respecto a sus contrapartes clásicas; sobre todo en VR y con usuarios con necesidades especiales.

Mayor inmersión en entornos VR

NUIs como el control mediante gestos de manos recrean la experiencia de tocar un teclado auténtico, lo que facilita la interacción directa con estos. Esto da una sensación

de presencia e involucramiento ni remotamente comparables a los modelos tradicionales con el mando. [26]

Mayor accesibilidad

Los teclados virtuales basados en NUI pueden ser especialmente útiles para personas con discapacidades o dificultades para usar pantallas táctiles o mandos tradicionales. Por ejemplo, mediante el uso de sensores de movimiento, giroscopios o el habla, se puede permitir introducir caracteres sin pulsar teclas activamente. [27]

Simulación más realista

Las NUI consiguen una simulación más realista del uso de un teclado físico mediante la utilización de motores físicos y la detección de impactos entre dedos virtuales y teclas. No solo potencia la experiencia del usuario, sino que también tiene el potencial de incrementar la exactitud y la satisfacción al interactuar con el teclado virtual. [26]

Conclusión

Las NUI en teclados virtuales aportan mayor inmersión, accesibilidad y realismo, haciendo que la interacción en entornos VR sea más natural y cómoda para variedad más amplia de usuarios.

2.6. Retos de accesibilidad en realidad virtual

2.6.1. Barreras actuales en VR para usuarios diversos

Dificultades físicas

La realidad virtual actual plantea serios problemas para individuos con movilidad restringida o problemas motores. En numerosos sistemas de Realidad Virtual, se asume que los usuarios pueden usar ambas manos de manera óptima, esta suposición ignora a las personas con dificultades o barreras relacionadas con las manos. Más del 50 % de las aplicaciones evaluadas resultan inaccesibles para individuos con una sola mano, y ninguna proporciona alternativas de personalización para individuos con limitaciones físicas. Esto pone de relieve la importancia de crear nuevos paradigmas y de ajustar las interfaces a este tipo de individuos [28]. Además, la dependencia de entradas físicas precisas y rápidas puede dificultar la participación igualitaria de usuarios con discapacidades motoras, ya que las interfaces tradicionales carecen de flexibilidad para adaptarse a las necesidades individuales [29][30].

Para hacer frente estos obstáculos, se han sugerido métodos adaptativos fundamentados en el aprendizaje por refuerzo, que posibilitan que las interfaces de Realidad Virtual se adapten de manera dinámica a las habilidades del usuario, mejorando el reconocimiento de gestos y la retroalimentación háptica. Estos sistemas han probado incrementar la exactitud en la interacción y disminuir el tiempo requerido para realizar tareas, lo que aumenta la satisfacción y disminuye la carga cognitiva para los usuarios con problemas

motrices [29]. Sin embargo, la integración de estas soluciones aún no está generalizada en la mayoría de las aplicaciones comerciales.

Dificultades sensoriales

Las personas con dificultades visuales o auditivas, dos tipos de trastornos sensoriales, se ven cohibidos ante la Realidad Virtual debido a la necesidad de utilizar señales visuales y auditivas para la navegación e interacción. La escasez de opciones excluye a estos usuarios de experiencias inmersivas. Por ejemplo, la falta de feedback háptico o de interfaces multimodales complica la participación de aquellos que no son capaces de percibir correctamente los estímulos visuales o auditivos. [31] [30][32]

La implementación de feedback multimodal, que fusiona estímulos auditivos, vibrátiles y visuales, ha probado el incremento del control y la estabilidad en usuarios con carencias sensoriales o de equilibrio, como individuos con esclerosis múltiple. La incorporación de estos modos sensoriales puede minimizar los obstáculos y potenciar la experiencia global de accesibilidad en Realidad Virtual. Asimismo, es crucial personalizar las configuraciones sensoriales para reducir la sobrecarga sensorial y ajustar la experiencia a las necesidades personales, particularmente en usuarios neurodivergentes o con patrones inusuales de procesamiento sensorial. [33][34][32]

Recomendaciones y oportunidades

Para superar estas barreras, es esencial adoptar principios de diseño inclusivo que consideren la diversidad de capacidades físicas y sensoriales desde el inicio del desarrollo de experiencias VR. La cooperación con usuarios con discapacidades, la incorporación de tecnologías emergentes de asistencia (como interfaces de voz, inteligencia artificial y háptica avanzada), y la capacidad de adaptar la experiencia a las necesidades individuales son tácticas fundamentales para progresar hacia una Realidad Virtual realmente inclusiva y accesible.

2.6.2. Necesidad de accesibilidad en teclados VR

La escritura requiere precisión y buena respuesta visual/háptica

Para asegurar una experiencia gratificante y eficaz de la escritura en ambientes de Realidad Virtual, se requiere precisión y una respuesta visual y háptica apropiada. La ausencia de feedback tangible, como el que se recibe en teclados convencionales, puede obstaculizar la exactitud y rapidez de la escritura en Realidad Virtual. Por ello, se han desarrollado soluciones como teclados virtuales con retroalimentación háptica realista, utilizando guantes hápticos y microaltavoces para simular vibraciones similares a las de un teclado físico, lo que mejora la interacción y el rendimiento del usuario en entornos totalmente inmersivos. En cambio, la opción de situar el teclado en cualquier sitio dentro del entorno virtual facilita el acceso y la comodidad eliminando la necesidad de dispositivos físicos adicionales. [35]

La exactitud en la redacción también se basa en el diseño de la interfaz y en el modo de entrada utilizado. Por ejemplo, se ha demostrado que los teclados “tap” ofrecen una mayor facilidad de uso, menor carga de trabajo y una experiencia de usuario superior en

comparación con los teclados de gestos (swipe) en Realidad Virtual, además de promover una rápida curva de asimilación [36]. Otros métodos, como los teclados con metáforas de entretenimiento (como el teclado de batería), han demostrado ser eficaces y divertidos, aunque todavía causan cierto rechazo, por lo que necesitan mejoras futuras [37]. La incorporación de feedback háptico y visual, además de la personalización de la disposición y funciones del teclado, son componentes esenciales para mejorar la accesibilidad y exactitud en la escritura en Realidad Virtual. [7][35]

El problema de los idiomas no soportados: exclusión de grandes segmentos de población

La falta de compatibilidad con múltiples idiomas en los teclados VR constituye un impedimento para la inclusión digital. Esto provoca la exclusión de numerosos grupos de la población que hablan otro idioma al diseñado. La capacidad de adaptar estos modelos de teclados para acomodar diferentes lenguajes, caracteres especiales y atajos, mejora el acceso a usuarios de diversos idiomas y contextos culturales [7][38]. Además, los sistemas propulsados por aprendizaje automático tienen la capacidad de ajustarse a diversos lenguajes de señas, dialectos y estilos de usuario, simplificando la comunicación y el acceso a la información para individuos con limitaciones auditivas y visuales [39].

La adaptabilidad y personalización de los teclados VR además de incrementar la accesibilidad para individuos con discapacidades, también pueden permitir la inclusión de usuarios de diversas procedencias lingüísticas y culturales. Sin un respaldo apropiado para varios idiomas, las soluciones de VR corren el peligro de prolongar la marginación digital y restringir el efecto beneficioso de estas tecnologías en la sociedad. [39][7]

Conclusión

La accesibilidad en teclados VR requiere una combinación de precisión, retroalimentación visual y háptica, además de soporte multilingüe. La falta de soporte para diferentes idiomas puede excluir a grandes grupos de usuarios, por lo que la adaptabilidad y personalización de los teclados virtuales son fundamentales para una experiencia verdaderamente inclusiva en entornos de realidad virtual.

2.6.3. Cómo este proyecto aborda la accesibilidad

¿La razón de ser de este proyecto? Acabar con esas barreras invisibles que siempre han marginado a tantos colectivos. Nos rompe ver cómo la tecnología, aún hoy, excluye a gente por fallos de diseño que podrían evitarse.

Por eso quisimos crear una joya de accesibilidad: un teclado con un superpoder oculto. Entiende de forma nativa idiomas tan distintos como el ruso con su cirílico, el coreano con su hangul o hasta el griego con su alfabeto milenario. Así rompemos la gran barrera idiomática. Esta característica permite su utilización a un mayor número de individuos al no tener esta barrera idiomática.

La otra función que también apoya la inclusividad en cierta manera es la capacidad de funcionar con hand tracking. Esto afecta tanto a individuos que, por diferentes motivos, les sea difícil coger un mando como a personas que, ya sea por la edad o dificultades cognitivas, encuentren muy complicado aprender o recordar el uso de botones complejos.

2.6.4. Limitaciones actuales y posibles mejoras

Actualmente aún hay muchos aspectos en el proyecto que se podrían trabajar para mejorar aún más la accesibilidad. Por ejemplo, aún no se cuenta con la posibilidad de incluir tanto a personas con discapacidades visuales agudas, como usuarios a con discapacidades motrices severas o directamente sin las extremidades superiores.

Los dos grandes caminos para seguir para poder llegar a más gente son los siguientes:

- Primero, se podría implementar una tecnología de SpeechToText que se activara mediante comandos de voz, esto podría incluir a todos los grupos mencionados en el anterior párrafo eliminando la necesidad de ver o usar las manos de raíz.
- El segundo camino se trataría simplemente de aumentar el repertorio de idiomas que soportara el teclado.

2.7. Estándares y tendencias de mercado en VR

2.7.1. Estado actual del mercado de VR

Crecimiento del mercado de VR

El mercado global de la Realidad Virtual ha experimentado un crecimiento significativo en los últimos años, impulsado por el desarrollo tecnológico y la aparición de dispositivos más asequibles y accesibles para el consumidor. Se estima que el tamaño del mercado mundial de VR aumentará de 7,3 mil millones de dólares en 2018 a 120,5 mil millones de dólares en 2026, según un informe de Fortune Business Insights, citado en el artículo “Virtual Reality” de Isabell Wohlgenannt, Alexander Simons y Stefan Stieglitz [40], lo que refleja una tendencia de crecimiento acelerado y sostenido hacia 2024 y 2025. Este crecimiento está motivado principalmente por el sector de software de consumo, especialmente los videojuegos, aunque también se observa una expansión en aplicaciones empresariales y de marketing.[40] [41]

Aumento de dispositivos standalone

La proliferación de dispositivos standalone, como la serie Meta Quest, ha provocado una expansión masiva de este mercado. Estos dispositivos permiten experiencias inmersivas sin necesidad de estar conectados a una PC de alto rendimiento, reduciendo barreras como el coste y la complejidad técnica [40][42][43][44]. Los HDM standalone están transformando el acceso a la Realidad Virtual, promoviendo la adopción a gran escala y promoviendo la creación de sistemas leves, como el seguimiento por mano, que optimizan la usabilidad y la experiencia del usuario [44]. Para añadir más peso a estos datos, se ha visto, mediante las pruebas de experiencias en el metaverso con estos aparatos, que la intención de uso futuro aumenta, pero que la intención de compra también depende de otros elementos vinculados con la percepción de valor y la experiencia del usuario [43].

Factores adicionales y tendencias

- La adopción de la Realidad Virtual se ve afectada por la tecnología y por elementos psicológicos y sociales de los usuarios, tales como su inclinación a la inmersión y su percepción de éxito social. [42]
- La calidad gráfica y la capacidad de procesamiento continúan siendo superiores en sistemas vinculados a PC, sin embargo, los dispositivos independientes consiguen un rendimiento y experiencia aceptables a través de optimizaciones, sin comprometer la exactitud del seguimiento ni la facilidad de uso. [44]
- Compañías de varios sectores, tales como la educación, la industria automotriz y la salud, están investigando aplicaciones de Realidad Virtual más allá del entretenimiento, lo que incrementa la cobertura y el potencial del mercado. [40] [41]

En resumen, el mercado de VR está en una fase de crecimiento acelerado, impulsado por la universalización de dispositivos standalone y la diversificación de aplicaciones, con una proyección de expansión significativa para 2025 y 2026. [40][43][44]

2.7.2. Estándares emergentes en VR

OpenXR

Como ha pasado en muchos productos tecnológicos anteriormente, uno de los principales desafíos históricos en el desarrollo de aplicaciones VR y AR ha sido la fragmentación de dispositivos y plataformas. Cada fabricante implementaba sus propias APIs y SDKs, lo que conllevaba hacer muchas versiones de un mismo proyecto para funcionar en cada servicio.

Para solucionar esto, el grupo Khronos —organización responsable de estándares como Vulkan y OpenGL— lanzó en 2019 OpenXR, abierta y gratuita, para unificar el desarrollo XR y no tener que hacer tantas versiones; una sola valdría para todo. Actualmente, la mayoría de gigantes del ecosistema XR, como Unity, Unreal, Godot, Blender, Valve, Microsoft y muchos otros, ya han adaptado este estándar internamente. [45]

Además, al incorporar características de tecnologías avanzadas como hand tracking, renderizado multivista, input dinámico y espacios anclados, su utilización se convierte en una elección estratégica, incluso para desarrolladores de Assets como este teclado multilingüe, ya que su compatibilidad asegura que el recurso pueda ser empleado de manera sencilla en visores actuales y venideros, sin la necesidad de reconfigurar controladores o ajustes particulares. [45]

Desde una perspectiva académica, la normalización sugerida por OpenXR ha sido vista como el elemento esencial para el desarrollo del metaverso y las experiencias XR interoperantes. [46]

Tendencia hacia experiencias sin controladores (“controller-free”)

Otra tendencia tecnológica del VR es el abandono de los mandos físicos en pos de experiencias “controller-free”. El mayor exponente de este apartado de tecnología es la

tecnología de hand tracking, explicada anteriormente en el punto 2.5.2 y cuyas ventajas para este proyecto también se han mostrado en el punto 2.5.4.

Mediante cámaras integradas y sistemas de aprendizaje automático, los visores modernos (como las Meta Quest 2 y Quest 3) capturan el movimiento de las manos del usuario en tiempo real, interpretando gestos como pulsaciones, agarres o desplazamientos.

2.8. Selección y justificación de idiomas implementados

2.8.1. Criterios de selección

Al seleccionar los idiomas para este Asset, me basé en cuatro criterios principales que equilibraron lo ideal con lo práctico:

Primero, busqué idiomas con muchos hablantes y amplia distribución geográfica. Quería que el proyecto tuviera el mayor impacto posible en diferentes regiones del mundo.

Segundo, necesitaba variedad en las formas de escribir. Elegí deliberadamente idiomas que usan sistemas de escritura muy diferentes entre sí. Esto me permitiría probar cómo se adapta el teclado a retos técnicos distintos.

Tercero, tuve que considerar mis propios conocimientos. Seleccioné solo idiomas cuyas reglas de escritura podía aprender y entender bien dentro del tiempo disponible para el proyecto.

Por último, evalué cuidadosamente lo que podía lograr con mis habilidades técnicas y las herramientas que tenía. Algunos idiomas prometedores quedaron fuera porque requerían soluciones técnicas que estaban más allá de mis capacidades actuales o de lo que el software podía manejar.

En resumen, busqué un balance entre el alcance global, la variedad técnica, lo que yo podía aprender y lo que podía construir.

Aclaraciones especiales

En los siguientes apartados se detallarán todos y cada uno de los idiomas que alguna vez se han planteado e incluso se han intentado desarrollar en este proyecto.

No se mencionará ningún otro idioma que use el mismo teclado que los que se van a nombrar, como por ejemplo el valenciano o catalán, pero esto no restringe su utilización si el idioma así puede emplearlo.

2.8.2. Idiomas implementados

- **Español:** lengua romance con más de 500 millones de hablantes en el mundo y que utiliza una variante del teclado QWERTY. Lo he seleccionado por ser mi idioma materno y, por ello, el que tenía mayor facilidad de implementación.
- **Inglés:** lengua anglosajona por antonomasia, tiene el mayor uso en el ámbito tecnológico y científico de entre todas las lenguas. Utiliza el teclado QWERTY que hace de base a las demás variaciones.

- **Francés:** incluido por su presencia en múltiples continentes y por representar una variante del layout AZERTY, lo que implica desafíos específicos en cuanto a disposición de teclas.
- **Alemán:** Ejemplo del layout QWERTZ, con adaptaciones ortográficas como la ß y umlauts, lo que aporta complejidad a nivel de interfaz.
- **Griego:** introduce un alfabeto completamente distinto al latino, lo que permite validar la adaptación del teclado a otros sistemas fonológicos y gráficos.
- **Ruso:** sistema cirílico con el layout JCUKEN. Tiene fuerte presencia en Europa del Este y Asia, útil para probar la adaptación a alfabetos de estructura densa y uso frecuente de caracteres exclusivos.
- **Coreano:** su inclusión responde al interés por evaluar la capacidad del teclado frente a sistemas de escritura silábicos y de construcción modular como el Hangul, y además ejerce de precedente de la incorporación de lenguas asiáticas.

2.8.3. Idiomas descartados y motivos

- **Chino:** Mayor exponente de los idiomas asiáticos e idioma con más hablantes nativos del mundo, lo que lo convierte en una elección clara en el caso de querer extender el posible mercado. Este idioma se investigó y se hicieron pruebas técnicas; no obstante, al usar un teclado QWERTY para escribir en pinyin (sistema de escritura que utiliza el alfabeto latino para caligrafiar como se pronuncian las palabras), cosa que luego se traduce a chino mandarín (dialecto más extendido del chino), aparecía el gran problema de que Unity no permite, por el momento, el uso de librerías dedicadas a este propósito, que ya existen en C# base. La solución pasaba por crear manualmente un archivo o base de datos para poder realizar esta conversión, reto que técnicamente no era complicado, pero cuya extensión lo hacía completamente inviable. El chino es un idioma que cuenta con más de 500000 palabras; a esto hay que sumar el hecho de que hay una gran cantidad de términos que cuentan con la misma fonología que otros, por lo que, al escribir una palabra en pinyin, podían salir hasta 11 coincidencias exactas a elegir.
- **Japonés:** Idioma de las tierras niponas, que cuenta con tres tipos de escritura distintas: el kanji, de origen chino; el katakana, silabario utilizado para representar palabras que vienen de otro idioma y onomatopeyas; y el hiragana, silabario que se emplea en la escritura de palabras japonesas, partículas y desinencias verbales. Tiene un sistema homólogo con el chino; también utiliza un teclado QWERTY estándar en el que se escribe romanji, que es como el pinyin, pero en este idioma, y se muestran sus homónimos en katakana, hiragana o kanji a elegir. Como con el chino, este idioma se intentó y se descartó por el mismo motivo: la incapacidad de Unity de utilizar librerías que transformaran el romanji y el excesivo trabajo para crear la base de datos.
- **Árabe:** Este idioma presentaba el mayor aliciente económico en mi proyecto, ya que fui informado por parte de mi tutor de la alta demanda de software por parte de hablantes de este idioma y cuya alternativa, cito textualmente, “moría en el inglés”. No obstante, la extremada complejidad técnica de intentar un idioma que, para empezar, se escribe de derecha a izquierda, no tiene vocales, cuyas letras presentan

grandes cambios dependiendo de que las acompañe y finalizando con el hecho de que Unity no está nativamente preparado para estas características, supuso, muy a mi pesar, el descartamiento final de este.

2.8.4. Retos técnicos en la implementación

- **Soporte tipográfico y codificación Unicode:** Al contar con diversos idiomas no latinos como el ruso o el coreano, hubo que jugar con el uso de Unicode y adaptar la salida del TextMeshPro para que se renderizaran correctamente los caracteres.
- **Diseño y disposición del teclado:** Se han tenido que adaptar los layouts de cada teclado, cambiando la posición y cantidad de teclas, a su idioma correspondiente.
- **Reconocimiento de entradas con hand tracking:** Al utilizar esta tecnología, fue necesario el trabajo de la correcta identificación de los toques para mantener la precisión.
- **Gestión del ajuste del idioma y del tipo de teclado:** Se ha desarrollado una interfaz que simplifica la transición de idiomas o de tipos de teclado, sin entorpecer con la experiencia del usuario ni dificultar su inmersión.
- **Creación de sílabas del Hangul:** Dado que el coreano funciona con sílabas en vez de palabras, se ha requerido la implementación de un algoritmo que automatiza la transformación de las sílabas durante la escritura.

Capítulo 3

Requisitos, especificaciones, coste, riesgos, viabilidad

3.1. Requisitos

1. Requisitos Funcionales

- RF 1: Se podrá escribir en el teclado.
- RF 2: Habrá un teclado virtual que simule uno físico y un teclado plano.
- RF 3: Se podrá cambiar entre el teclado físico y el plano cuando se requiera.
- RF 4: Los teclados tendrán una versión funcional en siete idiomas establecidos.
- RF 5: Se podrá cambiar de idioma cuando sea pertinente.
- RF 6: Se podrá interactuar tanto con mandos como con hand tracking.

2. Requisitos No Funcionales

- RNF 1: El sistema debe ser desarrollado usando Unity.
- RNF 2: Lenguaje de desarrollo C#.
- RNF 3: El teclado plano será con UI Canvas.
- RNF 4: El teclado físico estará formado por “3D Objects”.
- RNF 5: Deberá de ser un Asset.

3.2. Especificaciones

A continuación voy a redactar las especificaciones de los requisitos siguiendo el estándar IEEE [47].

3.3. Especificaciones

A continuación voy a redactar las especificaciones de los requisitos siguiendo el estándar IEEE [47].

1. Introducción

- a) **Propósito:** El propósito de esta especificación es definir los requisitos del sistema para el desarrollo del teclado virtual multilingüe en Unity, proporcionando una guía clara y detallada para los desarrolladores y las partes interesadas.
- b) **Alcance:** Esta especificación cubre los requisitos funcionales y no funcionales del sistema, así como las interfaces de usuario, limitaciones conocidas y futuras extensiones posibles.
- c) **Definiciones, Acrónimos y Abreviaturas:**
 - **RF:** Requisito Funcional
 - **RNF:** Requisito No Funcional
 - **VR:** Realidad Virtual
 - **SDK:** Kit de Desarrollo de Software
 - **TFG:** Trabajo de Final de Grado

2. Descripción General

- a) **Perspectiva del Producto:** El sistema consistirá en un teclado virtual que simule uno físico y uno plano en un entorno de realidad virtual (VR). Los usuarios podrán cambiar estos teclados y seleccionar el idioma deseado para la entrada de texto.
- b) **Funciones del Producto:** El teclado virtual permitirá:
 - Cambiar entre teclado físico y plano.
 - Seleccionar entre siete idiomas disponibles.
 - Escribir correctamente en cualquiera de los idiomas.
 - Funcionar tanto con mando como con hand tracking.
- c) **Características del Usuario:** Los usuarios serán principalmente aquellos que desarrollen aplicaciones de realidad virtual en las que se requiera la entrada de texto, como aplicaciones educativas y de entretenimiento.

3. Requisitos Específicos

a) Requisitos Funcionales

- 1) **RF1: Se podrá escribir en el teclado:** El sistema debe permitir al usuario escribir en cualquiera de las versiones del teclado correctamente.
- 2) **RF2: Tipos de teclado:** El sistema debe contar con una versión que simule uno físico y otra que sea plano.
- 3) **RF3: Cambio entre Teclado Físico y Teclado Plano:** El sistema debe permitir al usuario cambiar entre un teclado virtual que simula uno físico y uno plano.
- 4) **RF4: Múltiples idiomas:** El sistema debe posibilitar la escritura en siete idiomas para ambos tipos de teclado.
- 5) **RF5: Selección de Idioma:** El sistema debe permitir al usuario seleccionar entre los siete idiomas disponibles para el teclado virtual.
- 6) **RF6: Interacción múltiple:** El sistema debe permitir al usuario interactuar con él mediante los mandos y el hand tracking.

b) Requisitos No Funcionales

- 1) **RNF1: Plataforma de Desarrollo:** El sistema tiene que ser desarrollado utilizando Unity como plataforma principal.
- 2) **RNF2: Lenguaje de Desarrollo:** El sistema debe ser desarrollado utilizando el lenguaje de programación C#.
- 3) **RNF3: Formación del teclado plano:** El teclado plano se tratará de un UI Canvas.
- 4) **RNF4: Formación del teclado físico:** El teclado físico deberá estar compuesto de “3D Objects”.
- 5) **RNF5: Tipo de proyecto:** El proyecto tiene que ser un Asset.

4. Otros Requisitos

- a) **Interfaces de Usuario:** El sistema debe incluir interfaces de usuario intuitivas y fáciles de usar para la selección de teclado, idioma y entrada de texto.

5. Apéndices

Apéndice A.1

3.4. Costes

En la figura 3.1 se aparece el diagrama de Gantt que muestra la planificación temporal de mi TFG.

En la figura 3.2 se presenta una plantilla que muestra la planificación de los costes económicos relacionados con TFG.

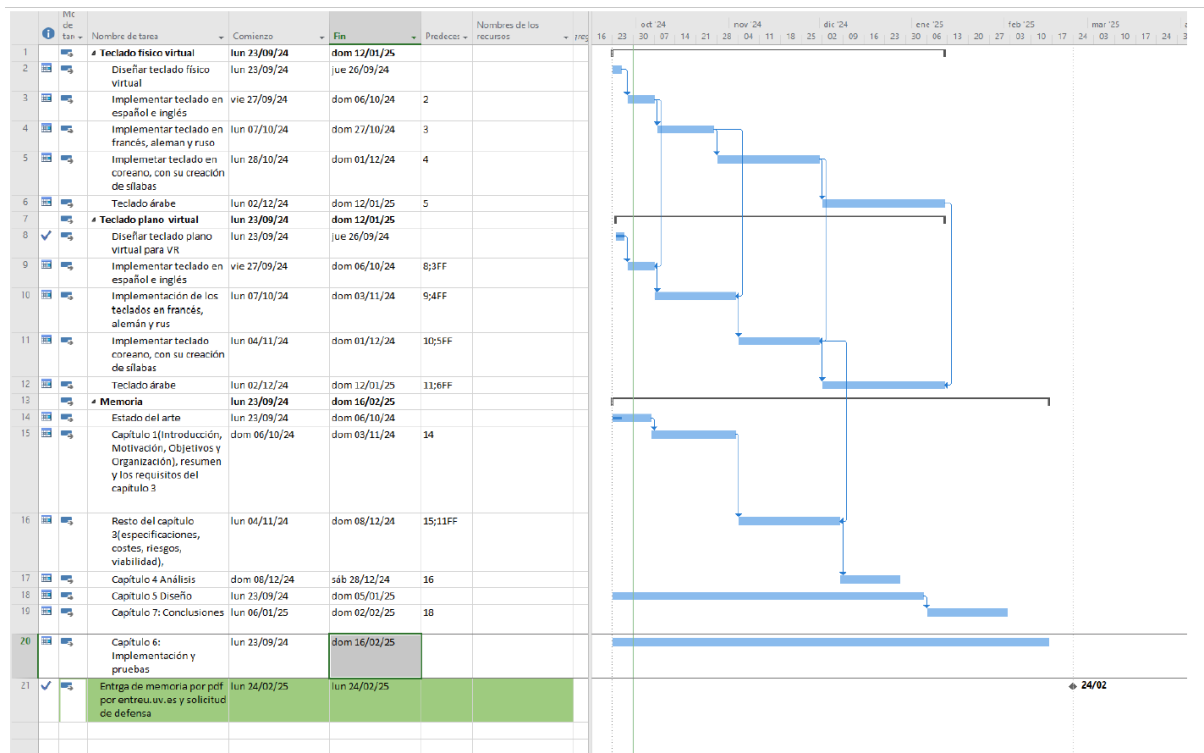


Figura 3.1: Diagrama de Gantt de los tiempos del TFG.

GASTOS DIRECTOS	
CONCEPTO	IMPORTE ANUAL
Yo	- €
Oculus Quest II	360,00 €
Ordenador nuevo(se rompió a la mitad)	2.296,00 €
TOTAL	2.656,00 €

GASTOS INDIRECTOS	
CONCEPTO	IMPORTE ANUAL
Luz	600,00 €
Internet	360,00 €
TOTAL	960,00 €

TOTAL	
	3.616.00 €

Figura 3.2: Plantilla de los costes económicos relacionados con el TFG.

3.5. Riesgos

En este punto, se analizarán posibles riesgos que puedan suceder en el proceso de desarrollo del proyecto. El estudio de estos se llevará a cabo para prevenir complicaciones que puedan retrasar o finalizar el trabajo de manera abrupta; a nadie le gustaría perder meses de trabajo (que es lo que dura el proyecto) por cualquier cosa que pueda ocurrir en este periodo.

En primer lugar, se van a disponer unos términos concretos para la clasificación de las situaciones. Se van a establecer dos tablas, una en función de la probabilidad y otra en función de la gravedad. Véase la tabla siguiente.

		IMPACTO EN EL PROYECTO		
		BAJO	MEDIO	ALTO
PROBABILIDAD DE QUE OCURRA (%)	ALTA [70 - 100]	RIESGO MEDIO	RIESGO ALTO	RIESGO INACEPTABLE
	MEDIA [35 - 70]	RIESGO BAJO	RIESGO ALTO	RIESGO INACEPTABLE
	BAJA [10 - 35]	RIESGO BAJO	RIESGO MEDIO	RIESGO ALTO

Figura 3.3: Tabla de clasificación de los tipos de riesgos relacionados con el TFG.

A continuación, y ya dada la clasificación de riesgos, se van a mostrar los que yo he encontrado o experimentado durante la realización del proyecto.

Riesgo	Probabilidad	Gravedad	Plan
Tecnologías			
Avería del ordenador	Baja	Alta	Plan de previsión: No hacer un uso excesivo de este equipo, ya sea con un empleo abusivo de programas o por demasiadas horas de uso. Plan de contingencia: como el peligro de este riesgo es perder el progreso, he decidido tenerlo todo guardado en un GitHub personal, por lo que, en caso de avería, podría usar otro equipo. Por otro lado, puedo reservar unos ahorros destinados a la reparación o adquisición de un nuevo equipo en caso de ser irreparable.
Avería de las gafas de realidad virtual	Baja	Alta	Plan de previsión: como con el ordenador, quiero tener ahorros reservados a la adquisición de unas nuevas. Plan de contingencia: reservar unos ahorros destinados a la reparación o adquisición de unas nuevas gafas en caso de que la avería fuese irreparable.
Fallo de conexión con la red	Media	Baja	Plan de contingencia: no estresarme y concentrarme en lo que puedo hacer sin tener que estar conectado a internet.
Personas			
Falta de motivación	Media	Alta	Plan de previsión: recurrir a métodos motivadores externos que potencien la sensación de validez al conseguir el propósito del proyecto. Plan de contingencia: buscar ayuda profesional para ayudarme a determinar cuál es la fuente de este problema y cómo intentar paliarlo.
Depresión	Media	Alta	Plan de previsión: realizar actividades que me hagan sentir bien e intentar no agobiarme, si no disfrutar del desarrollo del trabajo. Plan de contingencia: recurrir a ayuda profesional, si fuese necesario.
Enfermar	Alta	Baja	Plan de previsión: Cuidarme, llevar una alimentación adecuada y hacer deporte regularmente. Plan de contingencia: Ir al médico en caso de necesidad, reorganizar las horas de trabajo según mi capacidad en ese momento.

Empezar a trabajar	Alta	Alta	Plan de contingencia: Tener todo el programa hecho y que solamente falte redactar la memoria, cosa que puedo hacer en mi tiempo libre.
Continuar otros estudios reglados	Alta	Alta	Plan de contingencia: Tener una organización semanal muy estricta para poder abarcar todo.

Cuadro 3.1: Tabla de los riesgos relacionados con el TFG.

3.6. Viabilidad

3.6.1. Viabilidad técnica

- Plataforma de Desarrollo:
 - El uso de Unity como plataforma desarrolladora ofrece una gran robustez a la hora de desarrollar aplicaciones y Assets de realidad virtual, ya que es una plataforma bien establecida en este ámbito.
 - Al ofrecer Unity un soporte multiplataforma, este permite el desarrollo para distintos dispositivos de realidad virtual.
 - Disfrute de una considerable experiencia en el uso de esta plataforma.
- Lenguaje de Programación:
 - Se utilizará C# como lenguaje de desarrollo, tanto por ser el lenguaje principal de Unity como por contar con una gran cantidad de recursos y comunidades de soporte, así como lo necesario que ha sido su uso durante el transcurso del grado universitario.
- Bibliotecas y Herramientas Adicionales:
 - XR Interaction Toolkit: para la implementación de las funcionalidades en VR del proyecto.
 - OpenXR Plugin: plugin usado para la compatibilidad con múltiples dispositivos VR.
 - XR Hands: para permitir el funcionamiento del hand tracking en dispositivos habilitados para ello.
 - TextMesh Pro: utilizado para formatear el texto y permitir el uso de caracteres UNICODE en la interfaz de usuario.
- Pruebas y Depuración:
 - Al proporcionar Unity un entorno integrado para pruebas y depuración, facilita la identificación y resolución de errores durante el desarrollo.
 - Tengo permiso para probar el Asset en una clase del *Ciclo Formativo Superior de Técnico en Desarrollo de Aplicaciones Multiplataforma (DAM)* en el instituto IES Jaume II el Just de Tavernes de la Valldigna.

3.6.2. Viabilidad económica

- Costes del Desarrollo:
 - Licencias de Software:
 - Se hace uso de la versión gratuita del software Unity.
 - No se ha utilizado ningún plugin de pago del Asset Store.
 - Equipo de Desarrollo:
 - Todos los recursos humanos necesarios para el desarrollo, incluyendo programadores y diseñadores, se encuentran englobados en mi persona, por lo que no es una carga económica.
 - Estimación de horas de trabajo necesarias para cada fase del proyecto (diseño, implementación, pruebas).
- Costes de Infraestructura:
 - Hardware:
 - Ordenadores: he necesitado comprar un ordenador nuevo, por el precio de 2296,00€, puesto que se había roto el que estaba trabajando. Aparte de eso, no ha habido más gastos.
 - Dispositivos de realidad virtual para pruebas y demostraciones: he adquirido las Oculus Quest 2 por el precio de 360,00 €.
 - Servicios en la Nube:
 - Uso de GitHub como repositorio del proyecto.
- Análisis de coste-beneficio:
 - Beneficios potenciales:
 - Acceso a un mercado creciente de aplicaciones de realidad virtual.
 - Alto atractivo para la comunidad no anglosajona incluida en los idiomas descritos.
 - Monetización del teclado a través de la venta directa en Asset Store.
 - Retorno de Inversión (ROI):
 - No puedo poner un tiempo para que la inversión salga rentable debido a que no me es posible saber exactamente cuánto triunfará en el Asset Store, pero teniendo en cuenta el plan de vender el Asset a 20€, que es un precio muy competitivo, los gastos directos e indirectos mostrados en la figura 3.2, necesitaría vender unos 181 teclados.

3.6.3. Viabilidad temporal

- La planificación temporal se ha descrito anteriormente en la imagen 3.1; esto muestra el uso de un diagrama de Gantt para el monitoreo.
- La planificación ha sido realizada con gran margen respecto a la fecha límite de la entrega del TFG en caso de que surgiese algún problema.

3.6.4. Conclusión sobre la viabilidad

- Tras el análisis realizado, se puede concluir que el proyecto es viable.
- La plataforma de desarrollo, el equipo, los recursos necesarios y la planificación están alineados para asegurar el éxito del proyecto.

Capítulo 4

Análisis

4.1. Introducción

En este apartado, examinaremos a fondo el mecanismo del sistema justo antes de su activación. Tomando como referencia las especificaciones establecidas en la sección 3.1, distinguiremos a los participantes esenciales, los patrones de uso, las situaciones de manejo y los componentes del ámbito. Este conjunto de datos nos facilitará la edificación de una infraestructura estable para el diseño técnico subsiguiente.

4.2. Diagrama de casos de uso

Tal y como se indicó previamente, ya hemos detallado todas las necesidades del sistema, así que ahora les mostraremos un diagrama de casos de uso que comprende todas las funciones que ilustran dichas necesidades.

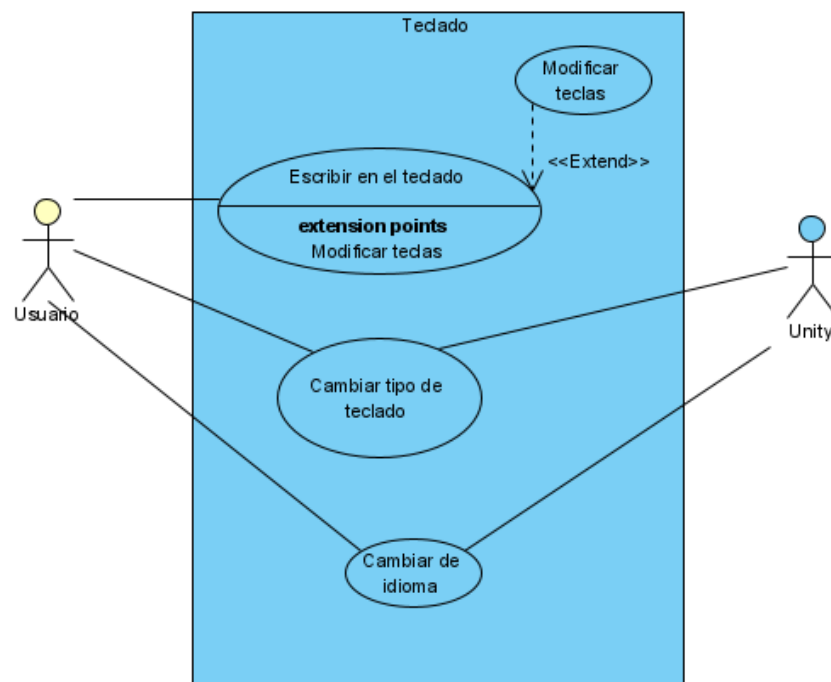


Figura 4.1: Diagrama de casos de uso

4.2.1. Descripción

Actores

- **Usuario:** Es el actor principal y más activo. Representa a la persona que interactúa con los teclados virtuales.
- **Unity:** Es un actor secundario que solo actúa en el arranque del sistema. Representa al motor de desarrollo utilizado.

Casos de uso identificados

1. Escribir en teclado:

- Es la acción estándar que se espera de cualquier teclado que se precie.
- Existe una interacción directa con el *Usuario*.

2. Cambiar tipo de teclado:

- Es la acción de cambiar entre el uso de teclados físicos o los planos.
- Existe una interacción directa con el *Usuario*.
- Existe una interacción directa con *Unity* al arranque del sistema que deja el teclado físico por defecto.

3. Cambiar de idioma:

- Es la acción de cambiar el idioma del teclado en uso.
- Existe una interacción directa con el *Usuario*.
- Existe una interacción directa con *Unity* al arranque del sistema para poner el idioma de tu SO o, en su defecto, el inglés .

4. Modificar teclas:

- Algunos idiomas necesitan modificar las teclas para poder utilizar acentos, mayúsculas y otras posibles características.
- Es una extensión del caso de uso de *Escribir en teclado* por lo que también requiere la intervención del *Usuario*.

4.3. Modelo del dominio

Seguidamente, expondremos un prototipo mental del dominio del sistema, en el que se precisarán las entidades fundamentales y sus conexiones, sin adentrarnos aún en los detalles de la ejecución. Este prototipo es valioso para comprender mejor las ideas abstractas que se emplean en el teclado virtual multilingüe.

Para ello, haremos uso de un diagrama de clases básico inicial que mostraremos a continuación. Los nombres de las clases muy probablemente se cambien en el futuro.

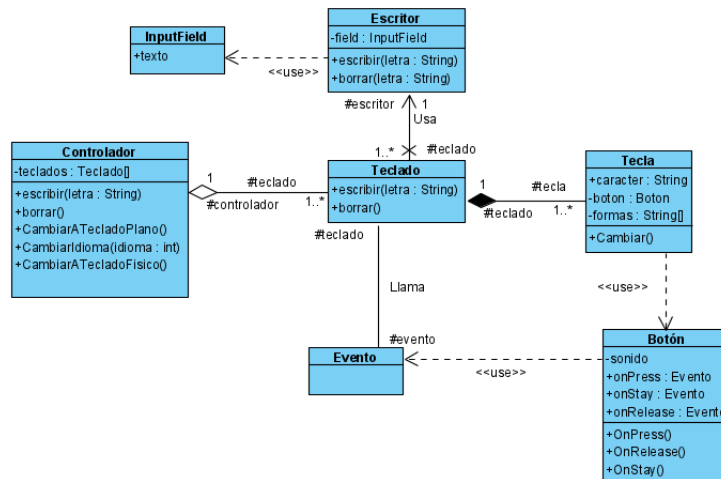


Figura 4.2: Diagrama de clases inicial

Todo esto es solo la idea preeliminar de como debería funcionar el teclado por encima, por supuesto no contempla la implementación final ya que aun no se tienen en cuenta las limitaciones o preferencias del motor usado. No obstante creo que es suficiente para comprobar el proceso que busco que siga el proyecto.

4.3.1. Descripción del diagrama

Clases

InputField

Es el sitio donde se escribe todo lo del teclado.

Controlador

Es la clase que controla a todos los teclado. es la encargada de escribir directamente en el *InputField*

Escritor

Es la única que se encarga de escribir directamente en el *InputField*

Teclado

Esta clase se encarga de controlar la lógica interna de cada teclado.

Tecla

Clase encargada de las funciones lógicas de las teclas de una tecla, así como guardar y cambiar el carácter que se escribe al pulsarla.

Botón

Encargado de las funciones físicas de las teclas, como hacer sonido cuando se pulsa, así como de escuchar eventos.

Evento

Clase básica de eventos que ejecuta funciones cuando se realizan.

Relaciones

- El *Escritor* hace uso del *InputField* para poder mostrar lo que se escribe.
- El *Controlador* tiene diversos objetos de la clase *Teclado* para controlar.
- El *Teclado* está formado por varios objetos de la clase *Tecla*.
- La *Tecla* usa al *Boton* para escuchar eventos.
- El *Botón*, como ya hemos dicho, escucha eventos concretos de la clase *Evento*.
- El *Evento* llama a funciones del *Teclado*.
- El *Teclado* hace uso del *Escritor* para realmente escribir.

4.3.2. Principios SOLID que se siguen

El Principio de responsabilidad única (Single Responsibility Principle - SRP)

“Cada clase tiene que tener una única responsabilidad y una sola razón para cambiar.”

Este principio dice que una clase debe tener solo una responsabilidad un motivo para cambiar, o sea, tener una única tarea.

Hasta cierto punto esto se cumple con las separaciones de la lógica del teclado en *Teclado*, la lógica de una tecla en *Tecla*, la parte física de una tecla en *Boton*, el control general en el *Controlador* y la escritura en el *escritor*.

El Principio Abierto-Cerrado (Open/Closed Principle - OCP)

“Las clases tienen que estar abiertas a la extensión pero cerradas a la modificación.”

Este principio nos indica que el código tiene que permitir añadir funcionalidades nuevas (abierto a la extensión) sin tener que modificar las clases existentes (cerradas a la modificación), reduciendo el riesgo de introducir errores en el sistema.

El uso de la clase *Evento* permite la extensión de comportamientos sin tener que modificar el código de *Boton*.

En *Tecla*, con el uso de la variable “formas”, permitiría la incorporación de varios idiomas sin necesidad de modificarse internamente. Por otro lado, y siguiendo la idea

de los idiomas, el *Teclado* está abierto a recibir extensiones necesarias para la correcta implementación de nuevos idiomas, pero sin modificar lo anterior.

En cuanto al *Controlador*, en este si que se tendrían que hacer cambios en el código para la implementación de nuevos idiomas, por lo que en este no se cumple.

El Principio de sustitución de Liskov (Liskov Substitution Principle - LSP)

“Las clases derivadas tienen que poder sustituir sus clases bases sin romper el comportamiento del programa.”

Esto explica que, si una clase derivada altera el comportamiento esperado de la clase base, puede provocar errores inesperados en otras partes del sistema. Es decir, si una clase A es la base de una clase B, cualquier instancia de B tiene que comportarse como una instancia de A sin romper el código.

En cuanto a este principio no puedo comentar nada, ya que, como se ve en el diagrama de clases, no hay herencia de ningún tipo, por lo que no hay clases derivadas de otras base que pueda o no respetar este. Por esto, este principio es como el gato de schrodinger, se cumple y no al mismo tiempo.

El Principio de segregación de interfaz (Interface Segregation Principle - ISP)

“Es mejor tener interfaces pequeñas y específicas que una única interfaz grande.”

Esta definición del principio explica que si una interfaz obliga una clase a implementar métodos que no necesita, significa que está mal diseñada. Es preferible dividirla en interfaces más pequeñas y enfocadas.

El Principio de inversión de dependencia (Dependency Inversion Principle - DIP)

“Las dependencias tienen que estar en abstracciones, no en clases concretas.”

Este principio significa que las clases de alto nivel no tienen que depender de las de bajo nivel. En cambio, todas tienen que depender de abstracciones (interfaces o clase abstractas) para hacer el código más flexible y fácil de adaptar.

La clase *Botón* usa objetos de tipo *Evento* para encapsular conductas. Esto constituye una modalidad fundamental de inversión de dependencias, dado que la lógica del botón se asigna a los objetos que llevan a cabo dichas acciones y no en unas concretas. Así, la clase *Botón* puede ser reutilizada y ampliada con facilidad sin tener que conocer los pormenores específicos de cada evento.

4.4. Análisis de comportamiento

- Descripción detallada de escenarios típicos de uso:
 - Escenario 1: Cambio de idioma en el teclado virtual
 - El usuario inicia la aplicación de realidad virtual.

- El teclado aparece con el idioma del sistema operativo si coincide con uno de los idiomas disponibles, si no, aparece en inglés.
 - El usuario selecciona el menú de configuración del teclado.
 - El usuario elige uno de los seis idiomas disponibles.
 - El teclado virtual cambia al idioma seleccionado.
- Escenario 2: Cambio de tipo de teclado (físico y plano)
 - El usuario inicia la aplicación de realidad virtual.
 - El teclado virtual aparece en su configuración predeterminada, que es el teclado físico.
 - El usuario pulsa el botón de cambio de teclado.
 - El teclado virtual cambia de una representación física a una plana.
- Ejemplos prácticos de cómo se utilizará el sistema:
 - Ejemplo 1: Uso en un juego de RV en varios idiomas
 - El usuario crea el juego y necesita usar el Asset.
 - El usuario selecciona el tipo de teclado (físico o plano) según su preferencia.
 - El usuario cambia la forma de cambiar de idioma para adaptarlo a su propio menú.
 - El usuario configura el juego y el teclado según sus necesidades.
 - Ejemplo 2: Redacción de datos en un programa de RV
 - El usuario crea el programa y se descarga el Asset.
 - El usuario pone el teclado virtual en el idioma deseado.
 - El usuario selecciona el tipo de teclado (físico o plano) según su preferencia.
 - El usuario configura la introducción de datos para usar el teclado.
 - Ejemplo 3: Simulación de mecanografía en RV
 - El usuario crea el simulador y se descarga el Asset.
 - El usuario pone el teclado virtual en el idioma deseado.
 - El usuario selecciona el de teclado físico o plano.
 - El usuario configura la introducción de datos para usar el teclado.
 - El usuario configura los objetivos mecanográficos necesarios.

4.4.1. Diagramas de actividad

A continuación voy a mostrar dos diagramas de actividad que resumen todas las acciones ocurren en el sistema. Cabe recalcar, para la comprensión de los siguientes diagramas, que he englobado todas las clases en una superentidad llamada *Teclado*, tal y como ocurre en el diagrama de casos de uso [4.2](#)

El primer diagrama representa el flujo de trabajo que puede realizar el actor *Usuario* al interactuar con el teclado, incluyendo los hechos de escribir y cambiar los idiomas o el tipo de teclado.

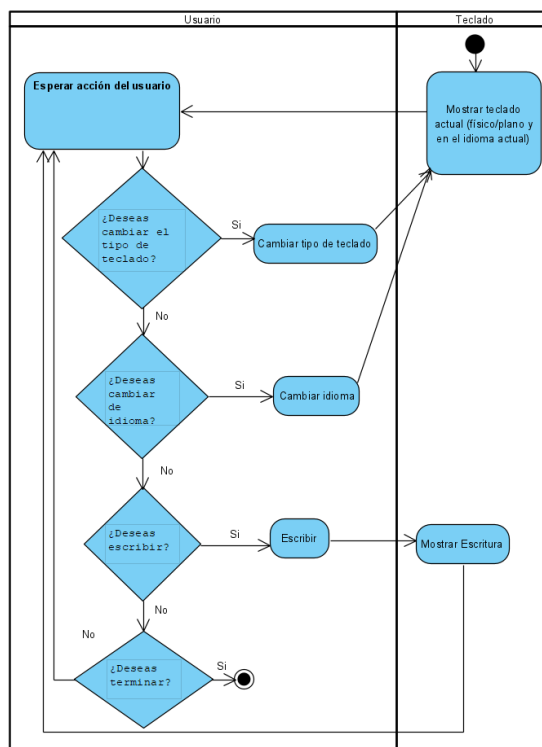


Figura 4.3: Diagrama de casos de uso

El segundo diagrama detalla las acciones del actor *Unity*, que se trata del propio motor usado para el desarrollo del Asset. Como se verá en la imagen, este solo se dedica a poner el teclado en un estado inicial estándar.

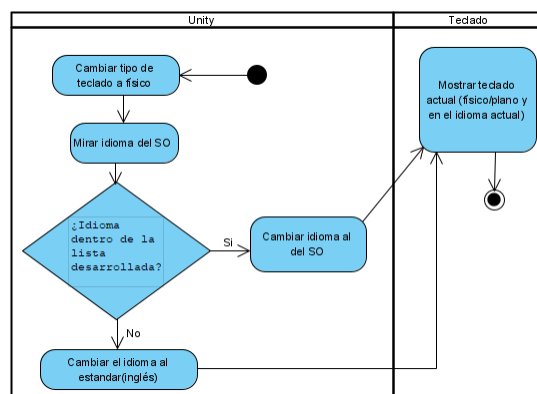


Figura 4.4: Diagrama de casos de uso

Capítulo 5

Diseño

5.1. Introducción

5.1.1. Objetivos

En esta sección trataremos de describir los objetivos principales del diseño del teclado virtual multilingüe para VR lo más detalladamente posible.

El primer objetivo se trata de garantizar la correcta escritura de los siete idiomas seleccionados y en los dos tipos de teclado, con soporte para cambios entre lenguas y tipos, por supuesto.

El segundo objetivo conlleva diseñar una interfaz intuitiva que facilite la interacción tanto con mandos como con hand tracking.

Como último objetivo, pero no por ello menos importante, es imprescindible conseguir que el software creado se integre en cualquier proyecto de Unity que utilice VR, ya que se trata de un Asset.

5.1.2. Relación con el análisis

El diseño va a proceder y extender el análisis previo (Capítulo 4), donde se identificaron casos de uso, se creó una base de modelo de dominio y se establecieron los requisitos técnicos.

En cuanto a los casos de uso, se consideró clave el cambio de idioma, la alternancia entre teclados físicos/planos y el soporte para caracteres especiales.

Por el lado del modelo de dominio, se van a extender las clases propuestas para la correcta integración con Unity y el seguimiento de los principios SOLID. También cambiarán el nombre de las clases para que, si alguien que no hable español lo quiere usar, sepa un poco de qué se trata. Más adelante se detallará todo este cambio.

Los requisitos técnicos ya se evaluaron y explicaron en la sección de viabilidad (Capítulo 3).

5.2. Diseño de la Arquitectura

5.2.1. Arquitectura del sistema

Estilo arquitectónico

El sistema pone en marcha una arquitectura de capas optimizada con patrones enfocados en eventos, creada específicamente para las exigencias singulares de la realidad virtual multilingüe. Incorpora tres perspectivas complementarias:

1. Arquitectura en capas:

- Capa de Presentación: Componentes Unity (UI Canvas, 3D Objects)
- Capa de Dominio: Lógica de negocio (gestión de idiomas, formación de caracteres)
- Capa de Infraestructura: XR Interaction Toolkit, OpenXR

2. Patrón MVC Adaptado:

- Modelo: Clases *Keyboard* y *KeyboardButton* (almacenan estado y reglas de idiomas)
- Vista: Prefabs de teclados físicos/planos y *TMP_InputField*.
- *KeyboardControlator* (orquesta la interacción).

3. Event-Driven:

- Eventos Unity (onPress, onRelease) para interacciones
- Delegados C# para comunicación entre componentes

Diagrama de Componentes

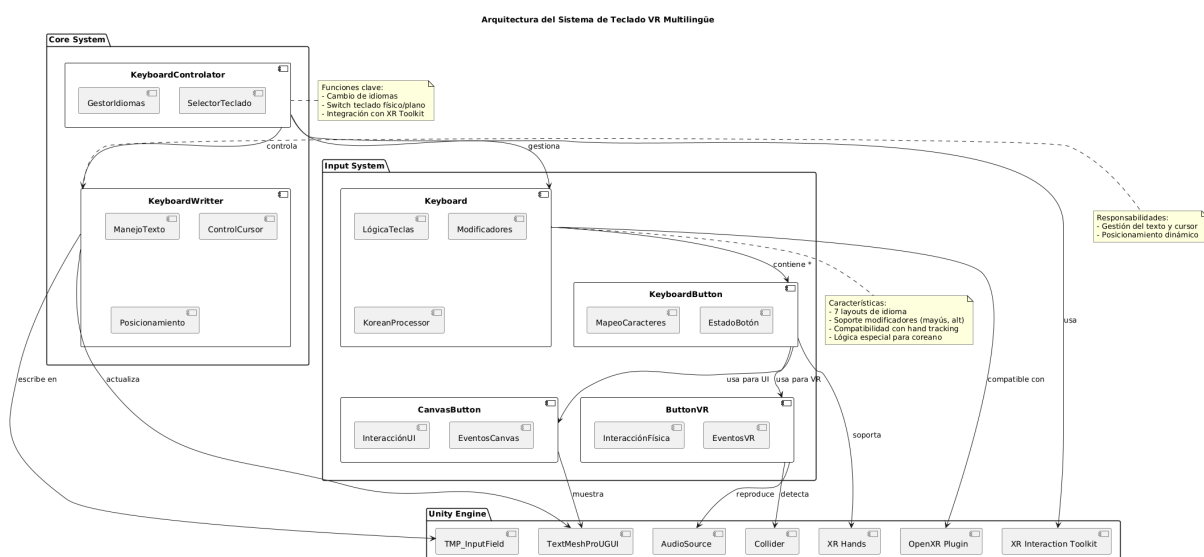


Figura 5.1: Diagrama de Clases Detallado - Relación entre clases

5.2.2. Diagrama de despliegue

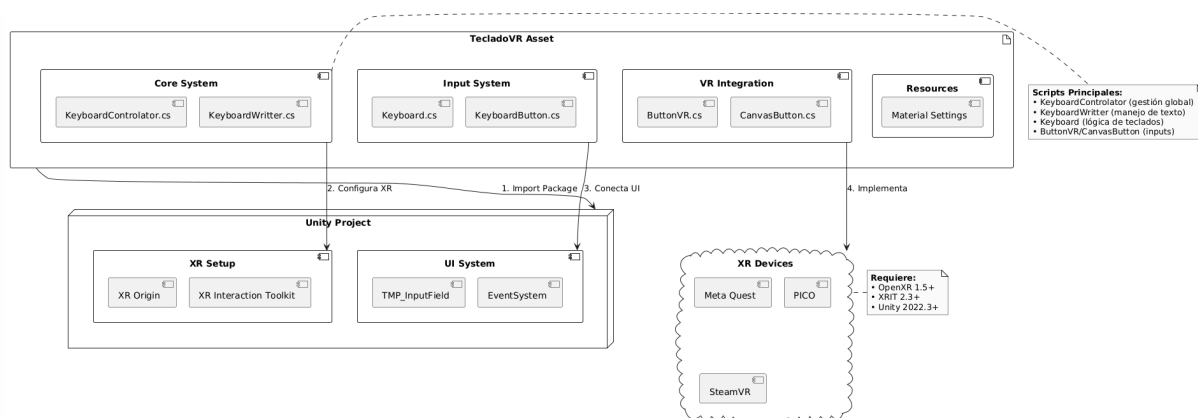


Figura 5.2: Diagrama de Clases Detallado - Relación entre clases

5.3. Diagrama de clases definitivo

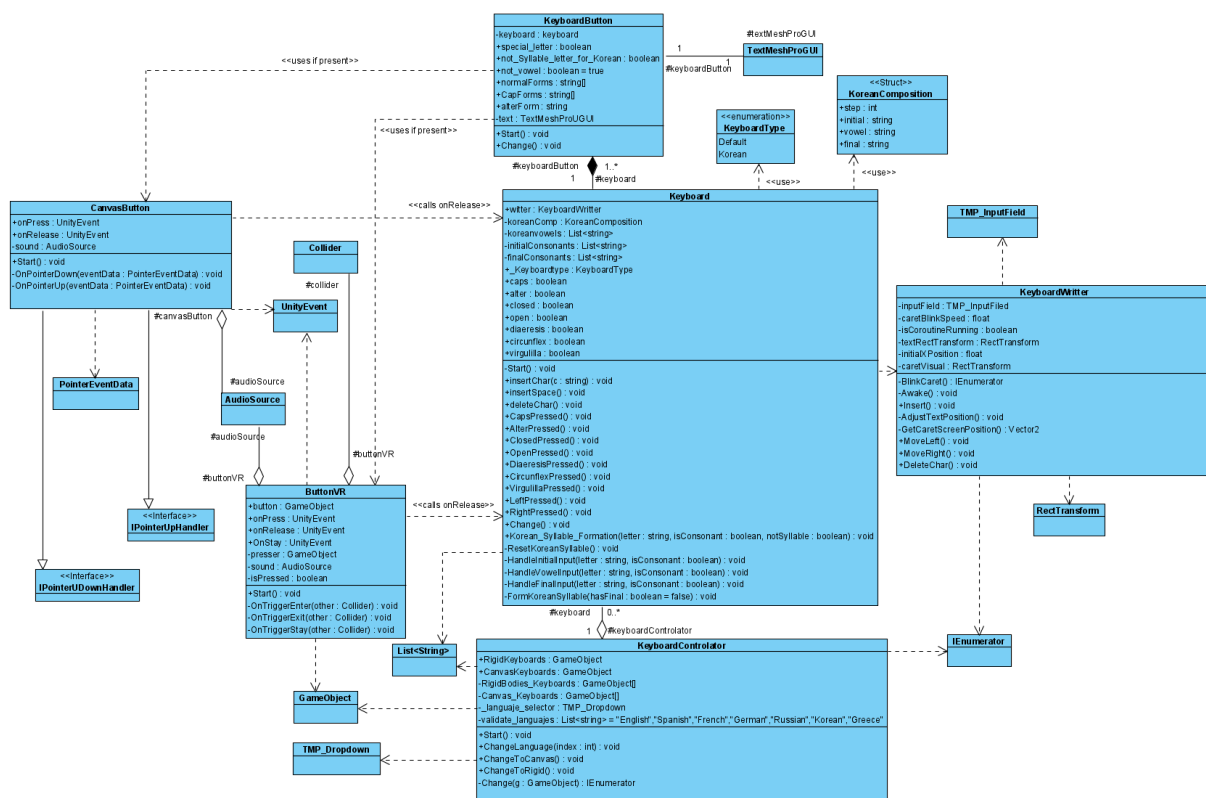


Figura 5.3: Diagrama de Clases Detallado - Relación entre clases

5.3.1. Descripción del diagrama de clases

1. Clases principales

KeyboardControlator

- Es una de las clases centrales del proyecto ya que se encarga de la coordinación entre todos los teclados.
- Maneja el cambio del tipo de teclado y de los idiomas.

KeyboardWriter

- Es la otra clase central del proyecto ya que es la encargada de escribir en el *TMP_InputField*, función principal de cualquier teclado.
- Maneja también la visualización del texto dentro del *TMP_InputField*.
- Centraliza la escritura al no estar esta capacidad desperdigada entre los teclados.

Keyboard

- Es el representante de cada uno de los teclados que hay.
- Maneja la lógica compleja necesaria en cada teclado, como por ejemplo “public void Korean_Syllable_Formation(string letter, bool isConsonant, bool notSyllable)” o “public void ResetKoreanSyllable()”. Centraliza estas funciones en una sola clase para evitar la creación de scripts diferentes para cada uno de los teclados, y que la actualización en caso de error o mejora no tenga que hacerse 14 veces.
- Llama a funciones *KeyboardWriter* para poder escribir.

KeyboardButton

- Representa una tecla individual de cada teclado.
- Maneja la lógica compleja de cada letra y preparado para cada idioma. Como con el *Keyboard*, es para no tener que hacer una clase para cada las teclas de cada teclado distinto y así que las correcciones y mejoras estén centralizadas.
- Asocia a un *TextMeshProGui* para actualizar por software la visualización de las letras y así ahorrar memoria al no tener que crear un botón para cada una de las distintas formas que podría salir una letra(por ejemplo cuando se pulsa para mayúsculas, acentos o ALTGR).

ButtonVR y CanvasButton

- Son los encargados de los eventos de las teclas de los distintos tipos de teclados. El *ButtonVR* se encarga de la interacción de los teclados físicos, mientras que el *CanvasButton* se encarga de la interacción de los teclados planos, que son creados con Canvas.
- Emiten eventos (onPress, onRelease) y controlan audio y colisiones.
- Los dos son usado indistintamente por el *KeyboardButton* para otorgar las funcionalidades necesarias a la llamada de eventos y llaman a funciones del *keyboard*.

2. Atributos y métodos relevantes

Idiomas

- El *language_selector* en *KeyboardControlator* incluye: “English”, “Spanish”, “French”, “German”, “Russian”, “Korean”, “Greece”. Esto es debido a que estos són los únicos idiomas que, por el momento, soporta el Asset.
- La nomenclatura de los idiomas está en inglés para facilitar el entendimiento del código a la mayor parte de los desarrolladores que quieran incluir este teclado. Esto presuponiendo que los desarrolladores saben inglés, al tratarse del idioma más usado por esta comunidad.

Método de cambio de idioma

- La función que realiza esta tarea es el “public void ChangeLanguage(int index)” situado en la clase *KeyboardControlator*, y se encarga de “activar” y “desactivar” los teclados correspondientes de los dos grandes grupos que hay (los físicos y los planos).
- Una utilidad muy interesante de este teclado es que automáticamente te pone el idioma de tu Sistema Operativo si está en la lista, en caso contrario se sitúa en el estándar, que es el inglés.
- Se ha instaurado un método manual de cambio de idioma mediante el uso de un *TMP_Dropdown* cuyo cambio de estado dispara un evento que llama a la función.

Métodos de cambio de tipo de teclado

- Al ser una de las primeros casos de uso desarrollados, esto se ha desarrollado en dos funciones distintas, cuando podría haberse hecho en una. Estas funciones són “public void ChangeToCanvas()” y “public void ChangeToRigid()” de la clase *KeyboardControlator*. Estas funciones activan o desactivan los objetos raices de los dos

tipos de teclados. Entre la desactivación del tipo actual y la activación del nuevo hay 1 segundo de espera, esto es posible gracias al hecho de que las dos funciones desactivan el respectivo objeto y luego llaman a la corrutina “IEnumerator Change(GameObject g)”, que hace la espera y luego activa el otro objeto.

- El Asset está configurado para que siempre salga con los teclados físicos al encender.
- Se puede cambiar de tipos de teclado manualmente al pulsar los botones correspondientes.

Soporte para coreano

- Toda la lógica para este apartado se encuentra en *Keyboard*. Por otro lado, es *ButtonVR* quien se encarga de llamar a estas funciones en concreto viendo si el “public KeyboardType _Keyboardtype;” está en “Korean”.
- Se poseen varias listas y variables encargadas de dar control a la formación de las sílabas usadas en este alfabeto.
- Las funciones específicas que se encargan de este idioma son el “public void Korean_Syllable_Formation(string letter, bool isConsonant, bool notSyllable)”, que se encarga de formar las sílabas del hangul mientras se escribe; el “public void ResetKoreanSyllable()”, que devuelve al estado base a las variables usadas para controlar la formación de las sílabas (cuando se da a espacio o se borra un carácter); “private void FormKoreanSyllable(string finalConsonant =)”, que convierte los componentes individuales en un carácter Hangul completo utilizando cálculos Unicode (código base 0xAC00); y “private void HandleInitialInput(string letter, bool isConsonant)”, “private void HandleVowelInput(string letter, bool isConsonant)”, “private void HandleFinalInput(string letter, bool isConsonant)” que son funciones para manejar la introducción de vocales o consonantes.
- El sistema sigue el proceso estándar de composición Hangul: consonante inicial + vocal + (opcional) consonante final, traduciendo esta combinación a su correspondiente carácter Unicode.

3. Relaciones entre clases

- Tanto *KeyboardControlator* como *KeyboardWriter* tienen asociaciones 1..* con *Keyboard* y sus representaciones visuales (*GameObject*).
- *Keyboard* contiene múltiples *KeyboardButton*.
- *KeyboardButton* usa *TextMeshProUGUI* para mostrar y actualizar los caracteres en las teclas y también es el *String text* de esta interficie lo que se pasa en la llamada a los eventos de escritura.
- *ButtonVR* y *CanvasButton* representan diferentes tipos de botones (dependiendo del tipo de teclado) y llaman a métodos en *Keyboard* cuando se activan los eventos. Los eventos específicos de escritura de caracteres son inicializados en *KeyboardButton*.

- Relaciones entre botones y eventos están claramente definidas con dependencias (<calls onRelease>), los otros tipos de acciones han sido creadas a mano.
- *CanvasButton* implementa funciones de las interfaces de *IPointerDownHandler* i *IPointerUpHandler* para el control de eventos. Estas funciones, a su vez, hacen uso del *PointerEventData*, ya que así lo requiere la instancia, pero internamente no se acaba utilizando.

5.3.2. Apéndices y aclaraciones

Como se puede ver, hay clases en el diagrama en las que solo he escrito el nombre, esto lo he hecho así ya que se tratan de clases propias de Unity que yo no he desarrollado, pero son necesarias para el correcto funcionamiento de este Asset.

El apéndice de este apartado se encuentra en A.2, en este se presentan algunas imágenes para ayudar a comprender mejor esta sección.

5.4. Diagramas de secuencia

5.4.1. Pulsado de tecla

En el próximo diagrama se mostrará el proceso que siempre se sigue cuando se pulsa una tecla cualquiera del teclado. Este quiero mostrarlo para evitar definirlo en los próximos diagramas, que ya de por si son largos.

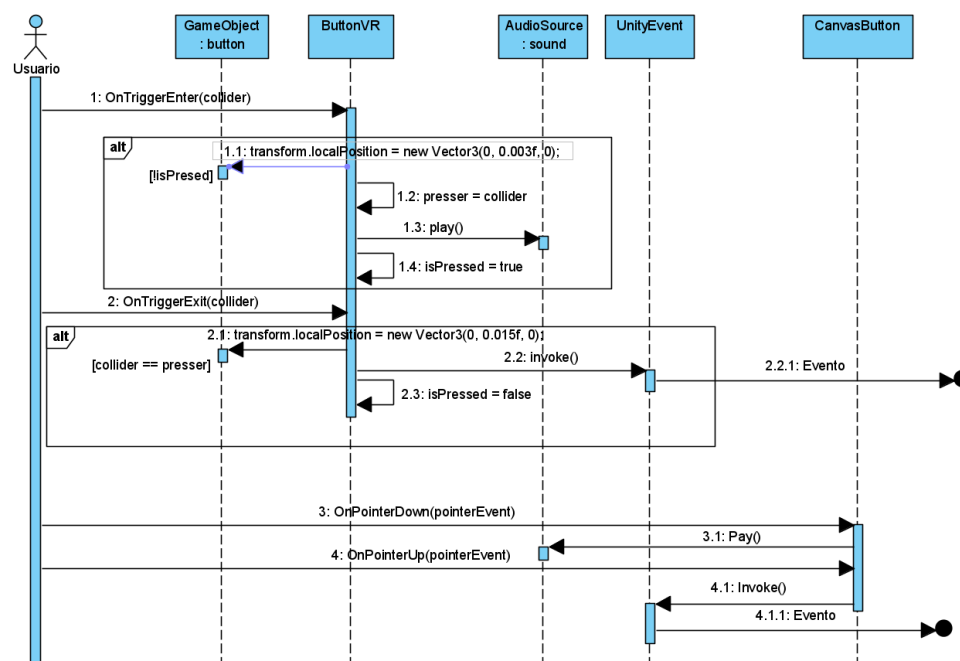


Figura 5.4: Diagrama de secuencia para cambiar de idiomas

Como podemos ver, he puesto *Evento* como señal perdida, se llama así para hacer referencia a cualquiera de las funciones que se pueden llamar al pulsar una tecla, y lo de la señal perdida es debido a que el “invoke()” del *UnityEvent* puede llamar a funciones de

varias clases distintas, por lo que no podía poner un agente, al no poder definir aun que hago.

Hay muchos actores involucrados en este proceso, desde un *AudioSource* para dar el sonido de pulsar teclas cuando estas se se accionan, hasta un “GameObject button” que movemos para ofrecer una experiencia simular el comportamiento de un teclado real.

Estos comportamientos se repiten en cualquier acción que involucre pulsar las teclas, por lo que en los siguientes diagramas resumiremos este proceso uniendo los dos botones (*CanvasButton* i *ButtonVR*) y el Usuario llamará a “OnTriggerExit(collider) / OnPointerUp(pointerEvent)” que usará el “invoke()” del *UnityEvent*, sin todo lo del sonido ni movimientos, etc.

5.4.2. Cambiar de idioma

En este diagrama de secuencia se muestra el cambio entre dos de los idiomas. Como se podrá observar, el tipo de dato de entrada en la función de cambio de idioma es un int, representados del cero al cinco, ya que se trata de un vector interno que contiene todos los idiomas. Esto no afecta al usuario ya que el interfaz visual que tiene, son los nombres de los distintos idiomas escritos en su sus grafías correspondientes mostrados en un dropbox.

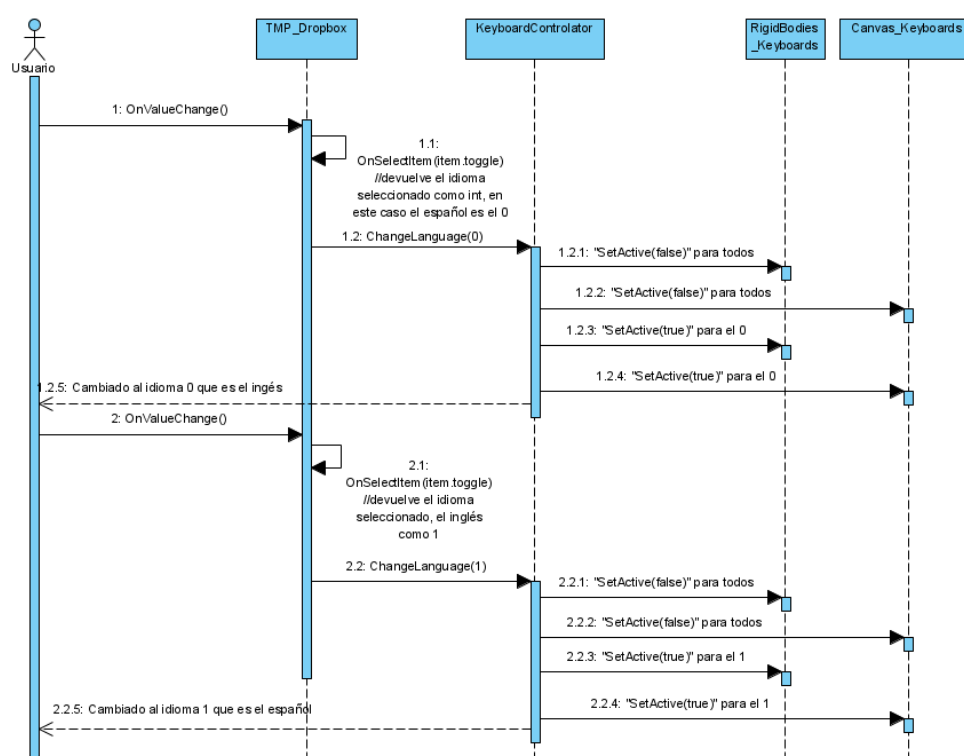


Figura 5.5: Diagrama de secuencia para cambiar de idiomas

5.4.3. Cambiar de tipo de teclado

En el próximo diagrama, se mostrará el cambio al teclado plano seguido de un cambio al teclado físico (el orden es debido a que, como ya he dicho, el teclado por defecto aparece como físico). El usuario solo debe tocar un botón predefinido para esto.

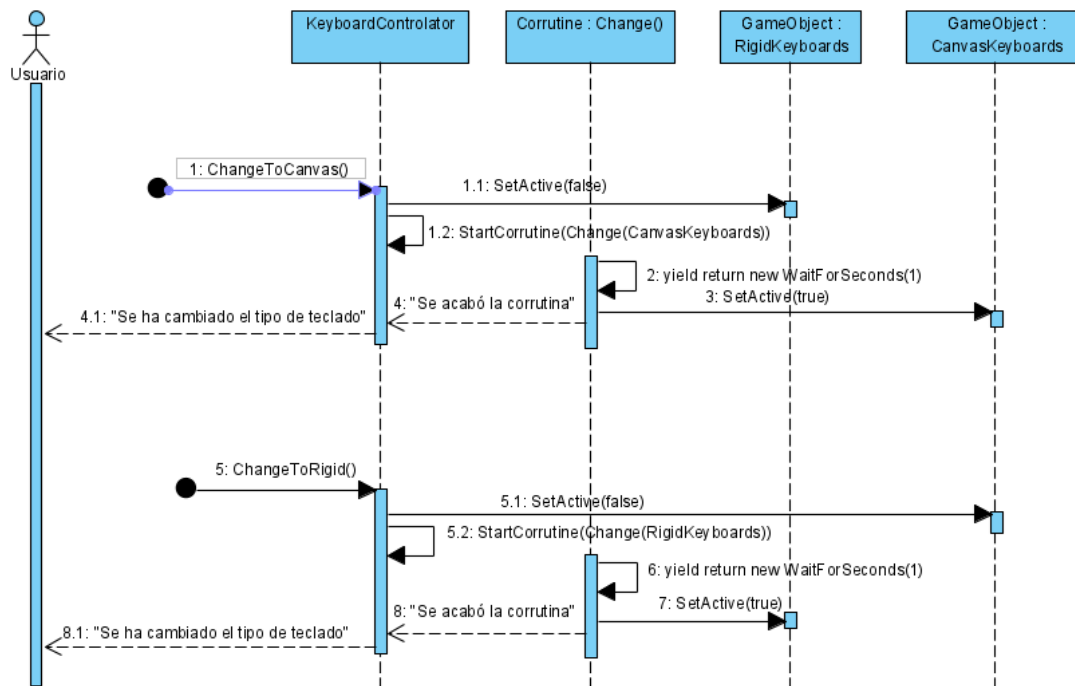


Figura 5.6: Diagrama de secuencia para el cambio de tipo de teclado

Como se observa, hay dos señales de entrada sueltas, estas hacen referencia a ese “Evento” del “invoke()” del *UnityEvent* explicado anteriormente, por lo que refiero a que todo el proceso del pulsado del boton también se realiza, sea cual sea el tipo de teclado.

La corrutina “Change()” es de dentro del *KeyboardController*, pero lo he separado para ayudar visualmente a entender este proceso.

5.4.4. Modificar letras

Esta característica se ha añadido aquí dentro ya que el modificar el teclado es un proceso indispensable para la escritura en los distintos idiomas, por lo que originalmente lo he incluido en el caso de uso de *Escribir en el teclado*.

Hay varias formas de modificar las letras, dependiendo del idioma. Todas estas se activan al pulsar el botón correspondiente de cada teclado, que cambia el estado de este, a continuación, todas llaman a “public void Change()” de *Keyboard*, que a su vez llama a la función tocaya en todos los *KeyboardButton* del teclado activo, que ya se encarga de cambiar las letras dependiendo del estado del teclado.

Vamos a ver el ejemplo de pulsar mayúsculas (en los idiomas que está disponible).

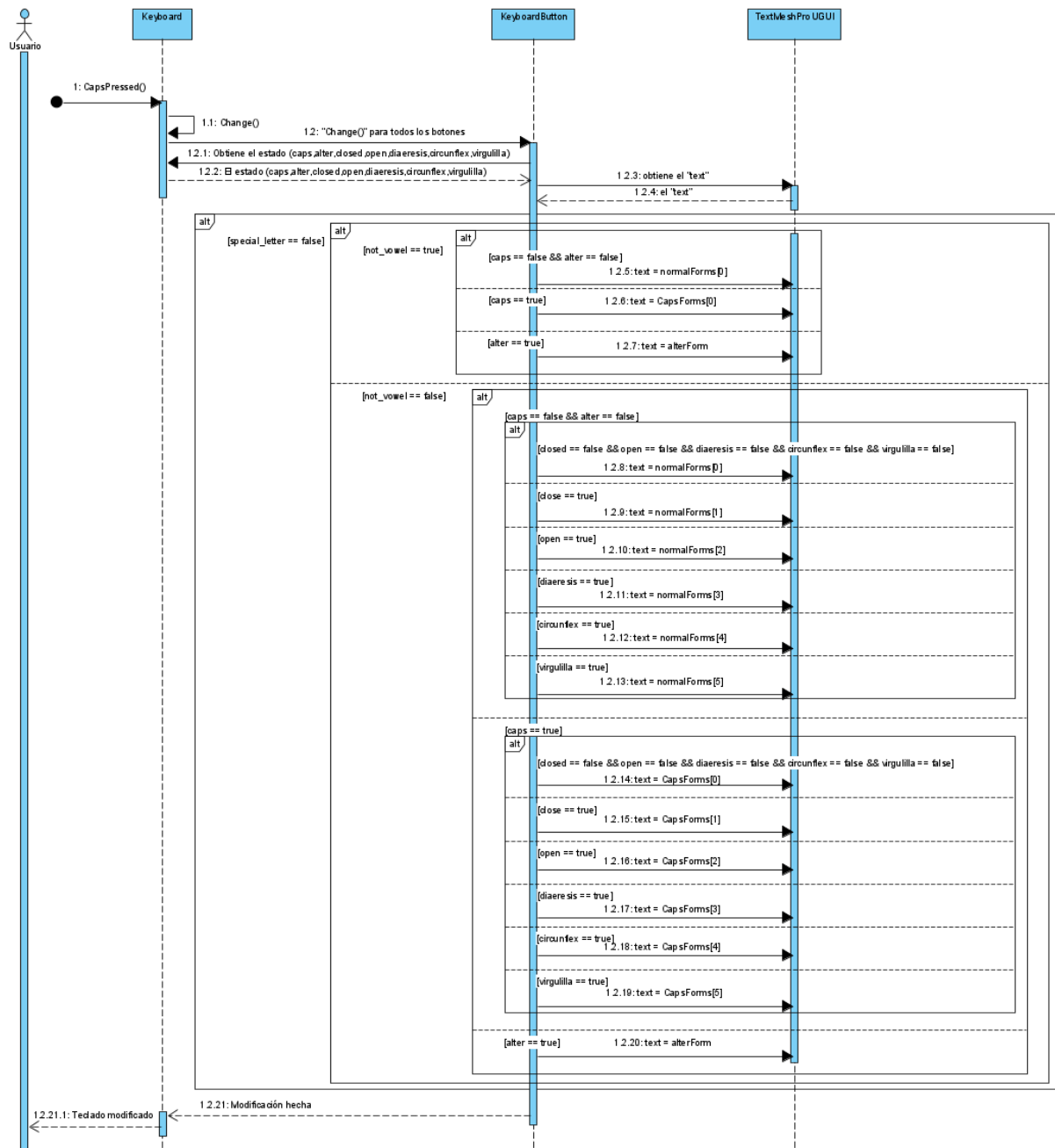


Figura 5.7: Diagrama de secuencia de la modificación de las letras de un teclado

Como podemos ver en la figura, el *KeyboardButton* comprueba cada vez que se le llama a la función “Change()” el estado del teclado, por lo que hace que el teclado que se use de igual, lo importante es que exista o no el botón que cambie este estado. En las secuencias de las otras maneras de cambiar el estado del teclado solo se diferencian de esta en cual es la función del *Keyboard* que llama el *CanvasButton* o el *ButtonVR* independientemente. Las acciones de poner los distintos acentos hacen lo mismo, pero con una función distinta del *KeyboardControlator*.

el “text” del *TextMeshProUGUI* es lo que las teclas muestran para saber que letra representan y lo que se cambia en esta acción.

Finalmente, la entrada de señal, como en el resto, representa que antes se realiza todo

el proceso del pulsado de una tecla, aquí da igual de que tipo de teclado se trate, ya que tanto en los físicos como en los planos, el proceso del cambio de letra es el mismo.

5.4.5. KeyboardWriter

Esta clase tiene varias funciones como “Insert(text)” o “DeleteChar”; todas relacionadas a la escritura, como ya he explicado.

Estos procesos tienen varios subprocesos, conllevan muchísimos componentes y da pie a un diagrama demasiado grande para poder verlo bien en la memoria, por lo que voy a desglosarlo en varios fragmentos. El diagrama total se verá dividido en las distintas funciones que tiene la clase, pero los fragmentos pueden mostrar más de una de estas divisiones.

Primer fragmento

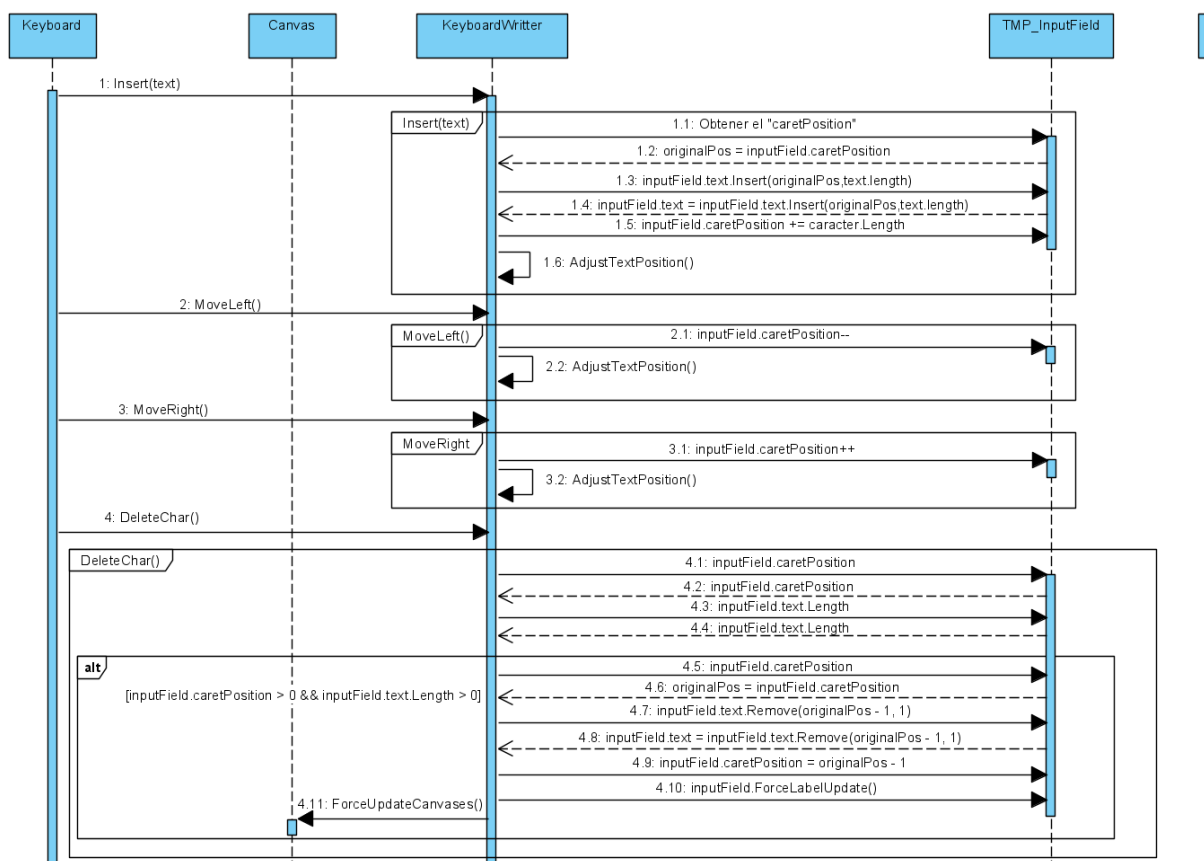
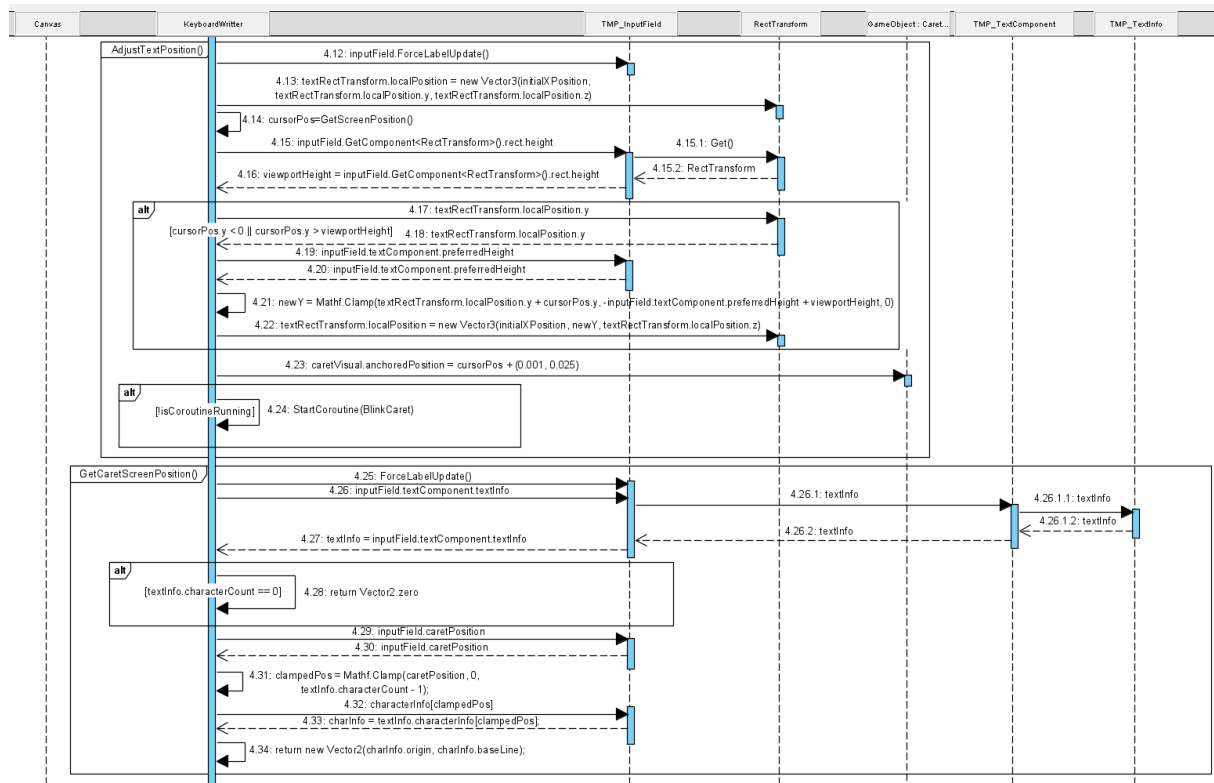


Figura 5.8: Primer fragmento del diagrama de secuencia de la clase *KeyboardWriter*

En este primer fragmento del diagrama mostramos el proceso de las funciones “Insert(text)”, “MoveLeft()”, “MoveRight()” y “DeleteChar”. Como se llega a observar, estas funciones hacen uso de otras que tiene la clase y que mostraremos en los siguientes fragmentos.

Segundo fragmento

Figura 5.9: Segundo fragmento del diagrama de secuencia de la clase *KeyboardWriter*

En este segundo, y último fragmento del diagrama, mostramos el proceso de las funciones “AdjustTextPosition()” y “GetCaretScreenPosition()”. Estas funciones se encargan de tanto el correcto emplazamiento del texto y el elemento visual (el “[SerializeField] private RectTransform caretVisual;”) para marcar la posición de escritura en el *TMP_InputField*.

Corrutina

Llegados a este punto, se me ha olvidado explicar el funcionamiento de la corrutina “BlinkCaret()”, que se encarga de hacer parpadear el elemento visual anteriormente nombrado, para simular típico cursor.

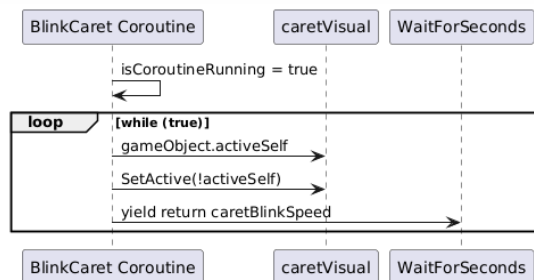


Figura 5.10: Diagrama de secuencia que muestra el proceso que sigue la corrutina BlinkCaret()

5.4.6. Insertar carácter

En este punto trataremos del proceso hasta la llamada a *KeyboardWriter*, que se ha destripado en el punto anterior.

Aquí se contemplará todo el proceso que sigue cualquier carácter, incluidos los coreanos y sus transformaciones.

Como al hacer el proceso genérico queda un diagrama muy grande, como el diagrama anterior, voy a dividirlo en trozos también.

Primer fragmento

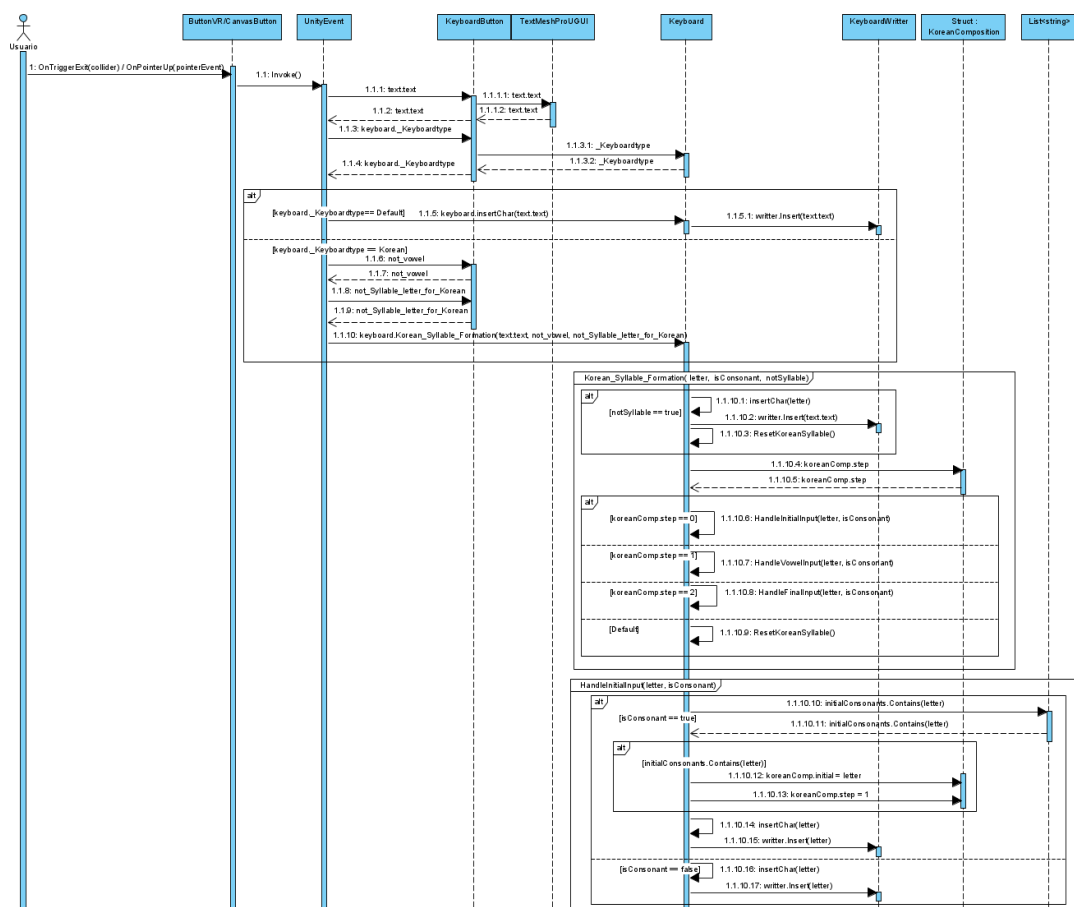


Figura 5.11: Primer fragmento del diagrama de secuencia que muestra el proceso para añadir un carácter

En este fragmento, lo primero que se puede ver como al pulsar el botón de un carácter, dependiendo que tipo de teclado sea, se llama a una función, con unas variables concretas, o otra. En un teclado normal añadimos el carácter y ya está, pero en el caso del coreano se tiene que empezar un proceso por el cual se tienen que formar sílabas en el caso de que se pueda.

En la función *Korean_Syllable_Formation*, que se componen de una consonante inicial, una vocal intermedia y de otra posible consonante final. Primero mira si el carácter es “silabable”, es decir, que es apto para la formación de sílabas, si no lo es simplemente

lo añadimos y devolvemos el estado al inicial, ya que es posible que ya hubiéramos empezado el proceso de formar las sílabas. Por otro lado si es apto, comprobamos el paso en la formación de las sílabas en el que nos encontramos, y llamamos a la función encargada de procesar cada paso.

En el primer paso de la formación de sílabas, la función encargada comprueba si la letra es consonante o no, es decir, si es vocal o consonante. En caso de ser vocal, una sílaba no puede empezar por una, así que simplemente añado la letra y ya, no toco el estado porque ya estoy al inicio. En el otro caso, es decir, cuando es una consonante, miro si es una consonante válida y, solo si lo es, avanzo de estado guardando esta letra como componente inicial de la posible sílaba, finalmente escribo este carácter.

Segundo fragmento

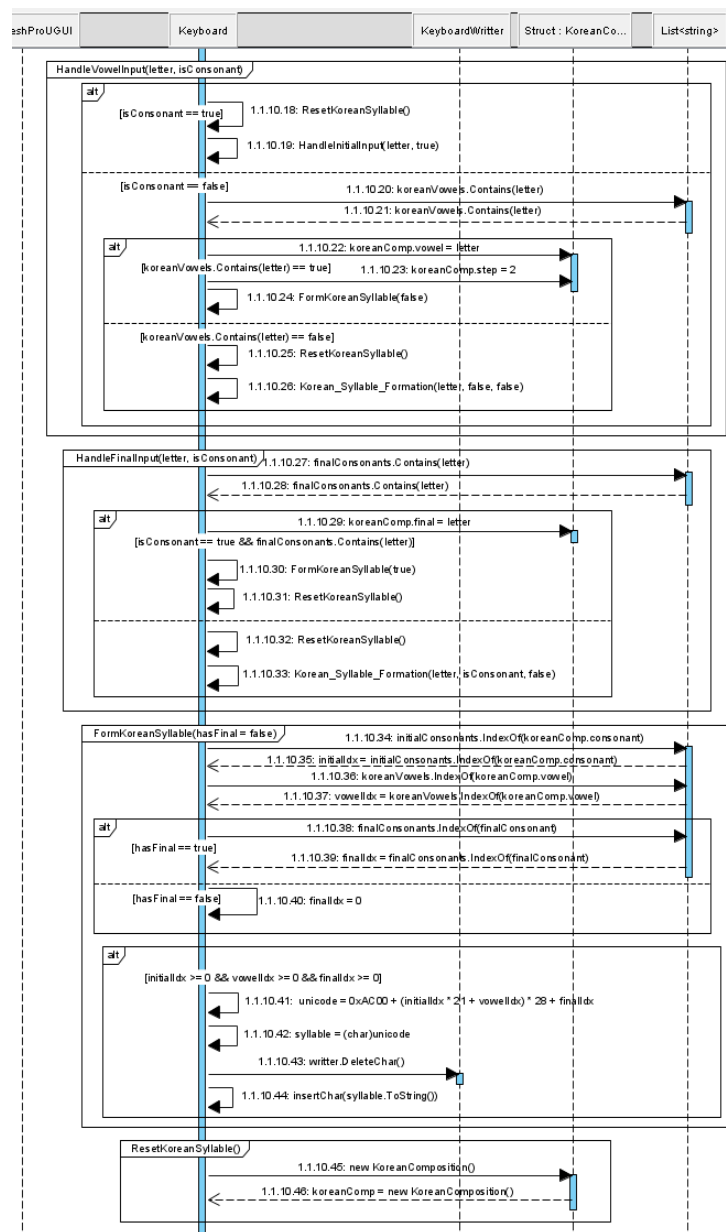


Figura 5.12: Segundo fragmento del diagrama de secuencia que muestra el proceso para añadir un carácter

Esta es la parte que resta del diagrama de arriba, se incluyen las otras funciones relacionadas a la formación de las sílabas con las que se escribe en Hangul.

En el segundo paso de la formación de las sílabas, la función encargada, como en la primera parte, mira que tipo de letra es, vocal o consonante, ya que se necesita otra vocal en este momento para formar una sílaba. Si es consonante, pos nada, reseteamos el estado y lo tratamos como si fuera del primer paso llamando a la función encargada de este. En las vocales se actúa como en se hacia con las consonantes del primer paso, es decir, comprobar si es una vocal válida para la formación; si es válida, pasamos de paso, guardamos el carácter como segundo componente de la sílaba y la formamos; en caso de no ser apto, reseteamos el estado y rellamamos la función *Korean_Syllable_Formation*.

En el paso final, comprobamos que sea tanto consonante, como que esta sea válida en esta fase. Si las dos comprobaciones salen positivas, guardamos esta letra como componente final de la sílaba, la formamos y reseteamos el estado, debido a que ya no se pueden añadir caracteres nuevos a la formación. En caso de que alguno de negativo, reseteamos el estado y rellamamos la función *Korean_Syllable_Formation*.

La función encargada de formar en si las sílabas, se le pasa una variable booleana, para saber si tiene o no consonante final que contemplar. Se sacan todos los índices de los caracteres que forman la sílaba, teniendo en cuenta que si no hay consonante final, se le da valor 0 por defecto a este índice, para no entorpecer en la creación de las sílabas. Si no hay errores en ninguno de los índices, se hace un pequeño cálculo matemático para sacar el código unicode de la sílaba a usar; luego se borra el último caracter, ya que se trata del estado anterior que queremos sustituir; y escribimos la sílaba.

El reseteo del estado de las sílabas lo lleva a cabo la función *ResetKoreanSyllable*, que simplemente crea un nuevo *KoreanComposition*, para sustituir el anterior.

5.4.7. Otras funciones del keyboard

En esta sección desglosaremos las funciones “LeftPressed()”, “RightPressed()”, “deleteChar()” y “insertSpace()” desde el pulsado de los botones correspondientes y hasta la llamada al *KeyboardWriter*.

En un principio iba a separar estas funciones restantes en diversos apartados, pero al ya haber cubierto la mayor parte de su contenido en el apartado del *KeyboardWriter*, he decidido juntarlos, ya que en el *Keyboard* cada una ocupa entre una y dos lineas de código.

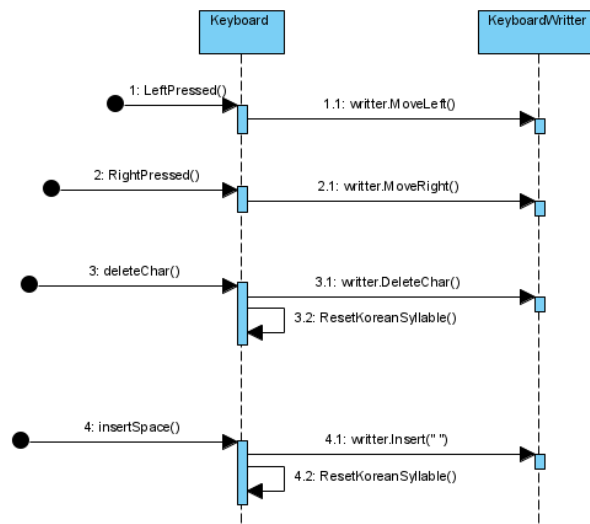


Figura 5.13: Diagrama de secuencia que muestra el proceso para añadir un carácter

Como ya se ha visto anteriormente, esos “mensajes encontrados” hacen referencia al “Evento” que enviaba el “invoke()” del *UnityEvent* cuando se pulsaba un botón. En cuanto al “ResetKoreanSyllable()”, se ha visto en el diagrama anterior.

5.4.8. Inicio del programa

Hasta ahora se ha visto todo lo que puede hacer el usuario, pero hay acciones que ocurren de manera automática cuando se inicia el programa para que todo realmente funcione. En esta sección detallaremos este proceso, del cual el usuario no tiene control y se encarga de realizar el propio motor Unity.

Hay bastantes elementos a incluir, por lo que optado en realizar varios diagramas, uno que incluya las clases cuya inicialización pueda plasmarse en un solo diagrama legible, y otros que muestren las clases restantes una a una, ya que necesitan su propio diagrama por tamaño.

Conjunto

Este primer diagrama combinará el proceso de inicialización que siguen las clases *Keyboard*, *KeyboardWriter*, *CanvasButton* y *ButtonVR*; que, como ya hemos dicho, son lo suficientemente cortos para poder juntarlos.

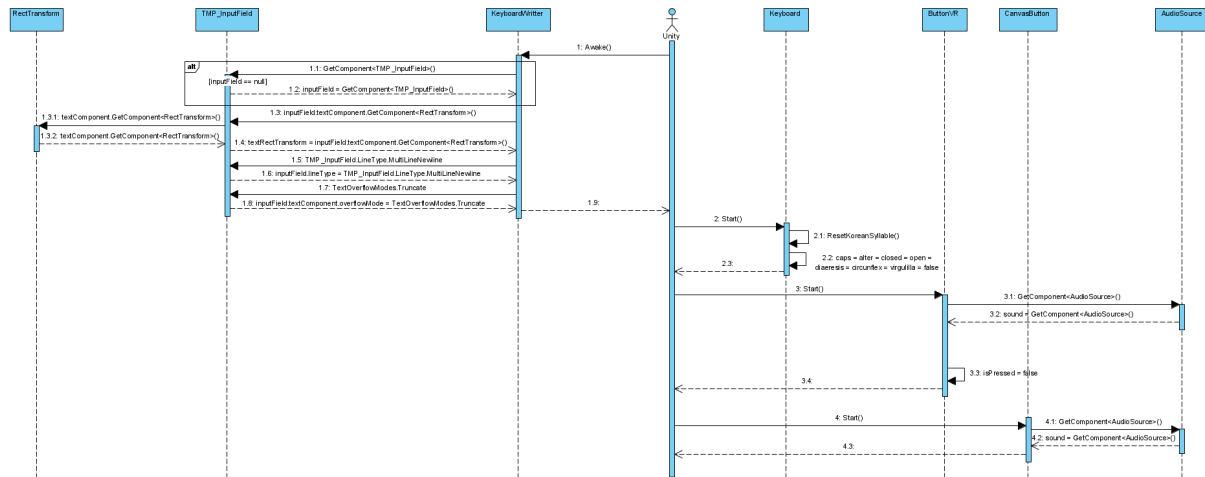


Figura 5.14: Diagrama de secuencia del inicio de los *Keyboard*, el *KeyboardWriter*, los *CanvasButton* y los *ButtonVR*

KeyboardControlator

Aquí describiremos el proceso de configuración inicial de la clase *KeyboardControlator*.

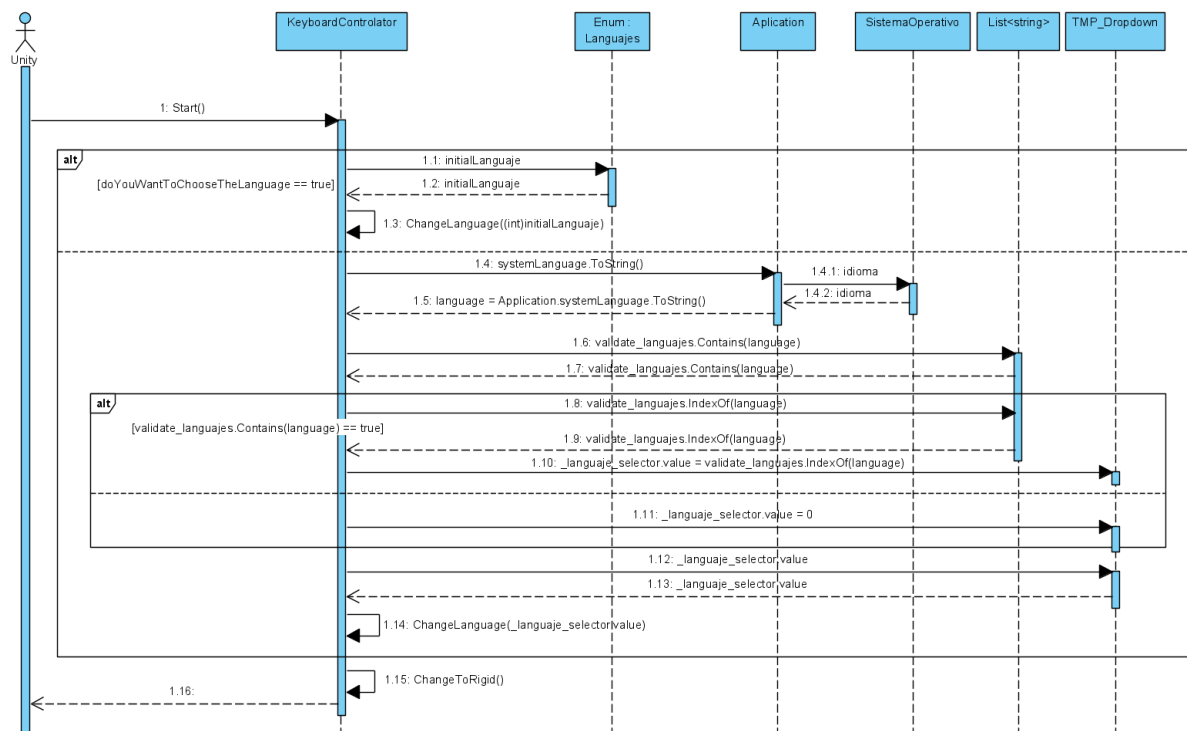
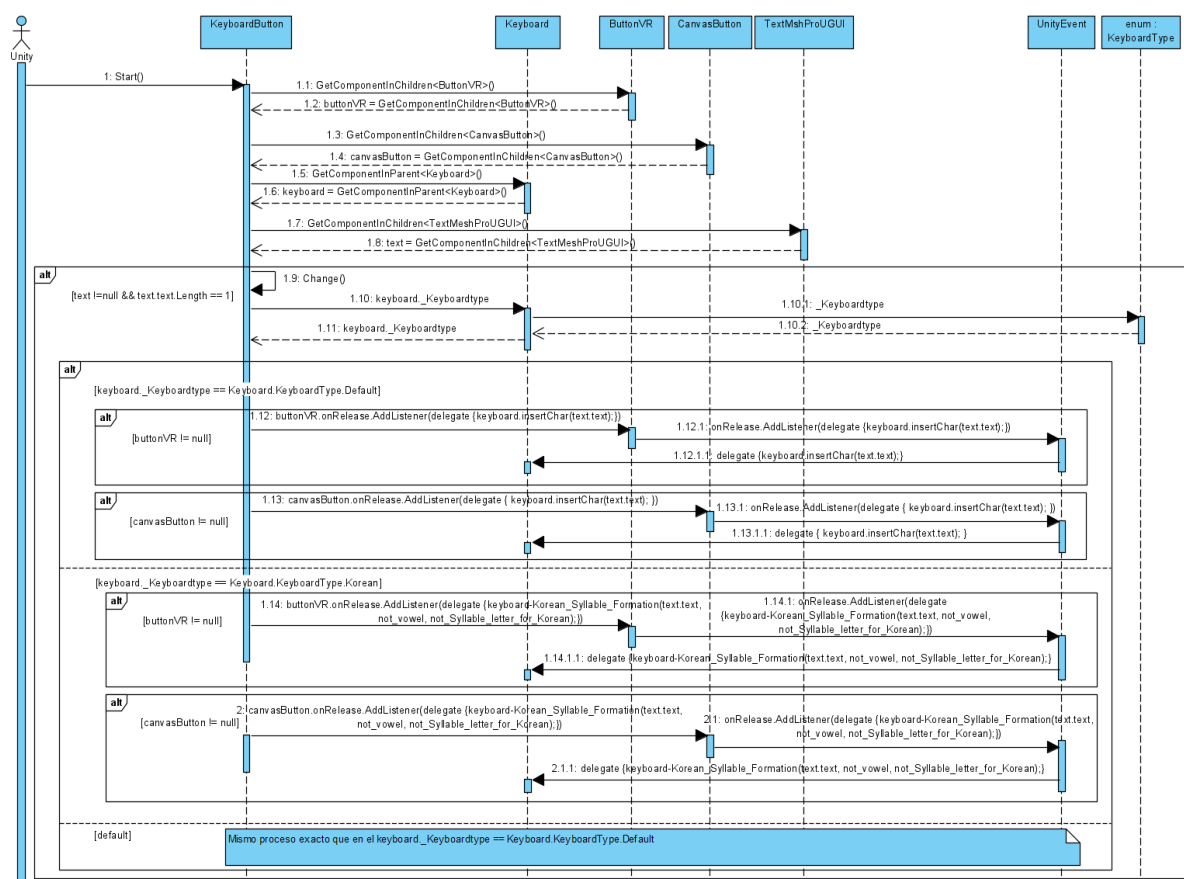


Figura 5.15: Diagrama de secuencia del inicio del *KeyboardControlator*

KeyboardButton

Aquí describiremos el proceso de configuración inicial de la clase *KeyboardButton*. Es, desde luego, el más largo de esta lista, e involucra muchas otras clases.

Figura 5.16: Diagrama de secuencia del inicio del *KeyboardButton*

Capítulo 6

Implementación y pruebas

6.1. Implementación

6.1.1. Tecnologías y herramientas

Primeramente, el motor, como ya se ha mencionado, es Unity, pero para el desarrollo de este se empleó más concretamente la versión 2021.3.9f1. En cuanto a los sistemas de interacción, destacamos dos, el XR Interaction Toolkit 2.3.0 para manejo de mandos VR y el XR Hands Package para soporte de hand tracking. Para la gestión de texto, se ha empleado el TextMesh Pro 3.0.6 con soporte Unicode extendido, que ya posee Unity, y fonts adecuadas. El plugin OpenXR 1.7.0, nos ha servido de backend multiplataforma. Finalmente, y como ya se ha explicado, se a mantenido un control de versiones con GitHub.

6.1.2. Desafíos técnicos específicos

Soporte de caracteres complejos

Uno de los mayores desafíos radicaba en el correcto renderizado de caracteres poco comunes como los del alfabeto cirílico, el griego y el coreano.

Idioma	Caracteres especiales	Soporte
Coreano	11,172 sílabas	100 %
Griego	24 letras + 5 acentos	100 %
Ruso	33 letras	100 %

Cuadro 6.1: Validación de caracteres especiales

Para ello se necesitó crear un Font Asset personalizado, que tardó 1 día y medio en procesarse, con una font que admitiera todos estos caracteres. En este caso en concreto se optó por utilizar la font NotoSansCJK-Regular, que también cuenta con el soporte de otras grafías asiáticas para posibles ampliaciones futuras.

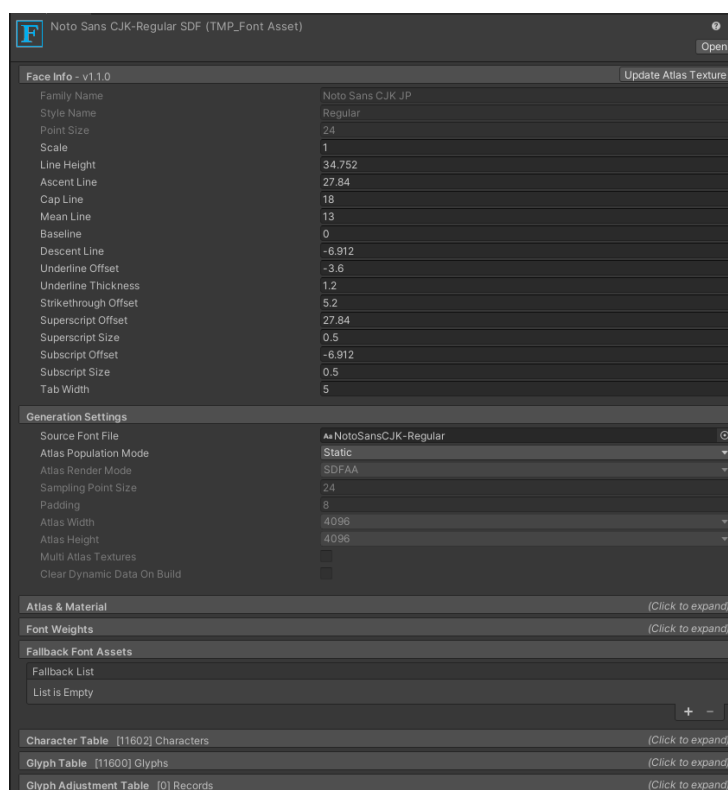


Figura 6.1: Font Asset personalizado

silabas del Hangul

El otro gran desafío se trato de desarrollar un código que admitiera la formación automática de las silabas del alfabeto hangul. Estas silabas se forman juntando una consonante inicial(hay 19), una vocal intermedia(hay 21) y una posible consonante final(28). La gran ventaja que se encontró es que las silabas en las tablas unicode se encontraban repartidas en bloques equitativos de combinaciones, por lo que se pudo desarrollar un algoritmo matemático que saca el índice correcto a partir del índice relativo de las letras que lo forman(se empieza por 0). Este algoritmo trata en coger el índice de la consonante inicial, multiplicarlo por 21, debido a las posibles combinaciones con las vocales, a esto sumarle el índice relativo de la vocal; al resultado se le multiplica por 28 por las combinaciones que tienen todas las silabas anteriores con las consonantes finales; a este resultado hay que sumarle la consonante final y da el índice relativo de la sílaba; a esto solo queda posicionarlo en la tabla unicode añadiéndole el índice real de la primera sílaba, que es el OxAC00, en exadecimal, o el 44032, en decimal.

```
// Calcular código Unicode
int unicode = 0xAC00 + (initialIdx * 21 + vowelIdx) * 28 + finalIdx;
char syllable = (char)unicode;
```

Figura 6.2: Fragmento del código con el algoritmo de la formación de sílabas hangul

Para ayudarme a sacar los índices correctos, me creé unas listas que consulto antes de aplicar el algoritmo.

```
// Listas de caracteres coreanos
private readonly List<string> koreanVowels = new List<string>()
{
    "ㅏ", "ㅑ", "ㅓ", "ㅕ", "ㅗ", "ㅛ", "ㅜ", "ㅠ", "ㅡ", "ㅟ",
    "ㅞ", "ㅠ", "ㅢ", "ㅤ", "ㅥ", "ㅦ", "ㅨ", "ㅩ", "ㅪ", "ㅫ", "ㅬ"
};

private readonly List<string> initialConsonants = new List<string>()
{
    "ㄱ", "ㄴ", "ㄷ", "ㄹ", "ㄺ", "ㄻ", "ㄼ", "ㄽ", "ㄾ", "ㄿ",
    "ㅀ", "ㅁ", "ㅂ", "ㅅ", "ㅆ", "ㅈ", "ㅊ", "ㅋ", "ㅌ", "ㅍ", "ㅎ"
};

private readonly List<string> finalConsonants = new List<string>()
{
    "", "ㄱ", "ㄴ", "ㄷ", "ㄹ", "ㄺ", "ㄻ", "ㄼ", "ㄽ", "ㄾ",
    "ㄿ", "ㅀ", "ㅁ", "ㅂ", "ㅅ", "ㅆ", "ㅈ", "ㅊ", "ㅋ", "ㅌ", "ㅍ",
    "ㅎ"
};
```

Figura 6.3: Fragmento del código con las listas

6.2. Validando el sistema: Mi experiencia práctica

Cuando llegó el momento de probar todo lo que había construido, me puse manos a la obra con un par de bebidas energéticas (sin azucar) y mucha paciencia. Lo primero que hice fue sentarme frente al visor VR y empezar a teclear como loco. Metí más de 150 caracteres raros a mano, uno tras otro, para ver si salían bien dibujados. Griego, coreano, cirílico... un montón de símbolos que parecían salidos de otro planeta. Recuerdo especialmente el problema con esa letra rusa *zo*" (esa que parece una *é* con sombrero) - ¡se veía toda aplastada! Pero después de jugar un rato con los ajustes de TextMesh Pro, por fin quedó decente.

Lo del coreano fue otro mundo. Me pasé una tarde entera probando combinaciones de consonantes y vocales como si fuera un juego de Lego lingüístico. "Vamos a ver, si juntamos H + A + N... ¿saldrá 'han'?" Y ahí estaba, la sílaba 10588 en la tabla Unicode, apareciendo como por arte de magia en la pantalla. Fue uno de esos momentos "¡jeureka!" que hacen que todo el esfuerzo valga la pena.

6.3. Pruebas de rendimiento

Te soy sincero, me daba un poco de miedo que todo fuera a explotar en plena demostración. Por eso me puse a observar cómo se comportaba el teclado en el mundo real. Durante las pruebas con mis compañeros, vigilaba como un halcón: ¿se atascaba al escribir? ¿Se cerraba de repente? Para mi alivio, aguantó bien el tipo. Eso sí, noté que en sesiones largas la memoria empezaba a respirar con dificultad, como cuando subes una cuesta con la bici cargada.

El gran susto vino con el Font Asset. ¡Catorce horas generando esa cosa! Me fui a dormir dejando el ordenador trabajando y al despertar... seguía ahí, tostándose como un pollo al horno. El resultado fue que la primera vez que abrías la app, tardaba lo suyo en cargar. Solución: ahora arranca antes que nada, mientras muestro ese logo molón que diseñé.

6.4. Lo que aprendí en el camino

Si hay algo que me quedó claro es que lo de las fuentes multilingües es un infierno. Nunca más quiero pasar otro día entero esperando que se genere un Font Asset. La próxima vez, buscaré fuentes prehechas o las partiré en trocitos. Por otro lado, TextMesh Pro fue mi salvación - sin él esto habría sido imposible.

Sobre la VR, descubrí cosas curiosas. La gente necesita un par de minutos para acostumbrarse al teclado flotante - al principio todos movían las manos como espantando moscas. Y las teclas tienen que ser más grandes de lo que crees, porque en el espacio 3D todo se ve más pequeño. Lo más importante: sin vibración en los mandos, tienes que poner buenas señales visuales, si no la gente no sabe si ha pulsado bien.

Al final, lo que más me llevo es que cada problema tenía solución. A veces tardabas en encontrarla, pero siempre aparecía. Como cuando a las 2 de la mañana se me ocurrió lo de precargar la fuente durante el inicio... ¡y funcionó!

6.5. Pruebas de usabilidad

6.5.1. Metodología SUS

Para este estudio, quise aprovechar que estoy cursando el Grado Superior en Desarrollo de Aplicaciones Multiplataforma. Después de hablar con varios profesores y compañeros, conseguí que 11 estudiantes de mi clase -con edades entre 18 y 28 años- probaran el teclado VR. La verdad es que me sorprendió lo dispuestos que estaban a colaborar. Casi ninguno tenía experiencia previa con realidad virtual (solo dos chicos habían usado algo antes), pero todos manejaban bien los ordenadores y teclados normales, claro, siendo estudiantes de informática.

Antes de empezar, fue importante que cada uno firmara el consentimiento informado [A.5](#). No quería dejar nada al azar en ese aspecto. Las pruebas fueron individuales, de unos 15 minutos cada una: primero usaban el teclado, luego rellenaban el cuestionario SUS [A.6](#). Ese formulario tiene su truco - las preguntas pares van "al revés", así que hay que estar atento al puntuar.

Cuando vi los resultados (Tabla 6.2), me llevé una grata sorpresa. Las puntuaciones fueron bastante altas en general. El gráfico de barras (Figura 6.4) muestra claramente cómo la mayoría se concentra arriba del todo. De hecho, si nos fijamos en el pastel (Figura 6.5), más del 70 % de mis compañeros dieron al teclado una nota de usabilidad .^{ex}celente". ¡Incluso uno llegó a 95 puntos! Solo tres estuvieron en la franja .^aceptable"(entre 68 y 84), y ninguno lo consideró inaceptable".

Eso sí, noté un patrón curioso: los dos compañeros que dieron las notas más bajas (70 y 77.5) fueron precisamente quienes más problemas tuvieron con los idiomas asiáticos. Uno me comentó después: "Lo del coreano se me hizo cuesta arriba". Pero incluso ellos reconocieron que el teclado funcionaba bien en general.

Personalmente, creo que estos números demuestran que el prototipo va por buen camino, especialmente considerando que casi todos eran nuevos en VR. Eso sí, la parte multilingüe necesita más trabajo, como veremos más adelante.

Cuadro 6.2: Puntuaciones SUS del teclado VR por alumno

Alumno	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	SUS
1	5	1	5	1	3	2	5	2	4	1	87.5
2	4	2	4	1	5	3	4	1	5	1	85.0
3	3	1	4	1	4	2	5	1	5	1	87.5
4	5	1	4	1	5	2	4	1	5	1	92.5
5	4	1	5	1	5	1	5	1	3	1	92.5
6	4	1	3	1	4	1	5	2	4	1	85.0
7	4	3	4	2	5	2	4	2	5	2	77.5
8	4	1	5	1	4	1	5	1	5	1	95.0
9	5	1	4	2	5	1	4	1	3	1	87.5
10	3	3	5	4	4	2	4	2	4	1	70.0
11	4	1	5	1	4	1	5	1	4	1	92.5

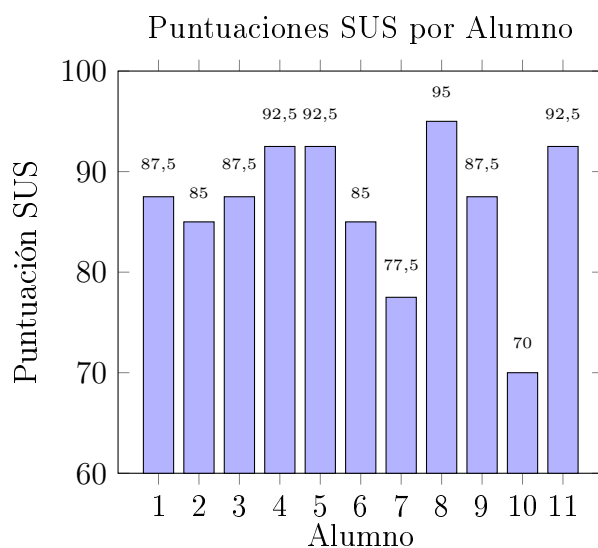


Figura 6.4: Distribución individual de usabilidad percibida.

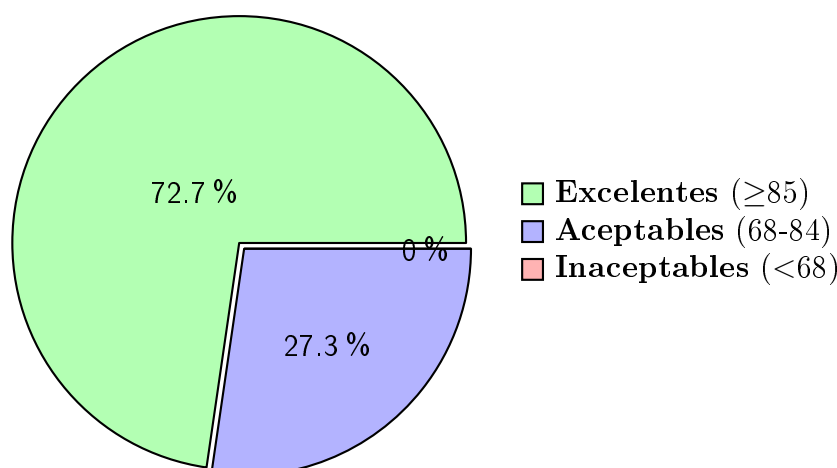


Figura 6.5: Distribución de usabilidad según SUS (11 alumnos).

Excelentes: 8 alumnos (72.7%), **Aceptables:** 3 alumnos (27.3 %), **Inaceptables:** 0 alumnos (0 %).

Análisis del Net Promoter Score (NPS)

El Net Promoter Score (NPS) me pareció especialmente revelador porque no solo mide satisfacción, sino lealtad real. Al preguntar a mis compañeros "*¿Qué probabilidad hay de que recomiendes este teclado VR?*", sus respuestas (Tabla 6.3) mostraron un patrón curioso. Casi todos dieron puntuaciones altas - entre 7 y 9 - pero con matices que los números solos no capturan completamente.

Fíjate en el gráfico de barras (Figura 6.6): ese vacío entre 0 y 6 salta a la vista. ¡Ni un solo detractor! Eso ya es alentador. Pero lo interesante fue cómo se agruparon las respuestas: cinco compañeros pusieron 8 (neutros), cuatro dieron 9 (promotores), y solo dos se quedaron en 7. Esto me hizo preguntarme: ¿por qué nadie dio el 10 máximo? En charlas posteriores, varios mencionaron que "para recomendar algo a un cliente" querían probar antes la versión definitiva.

Con esos números, el cálculo del NPS quedó en 45.5 (Figura 6.7). Según los estándares tecnológicos, se considera una puntuación positiva - claramente por encima de lo mediocre - aunque personalmente esperaba algo más alto. Al desmenuzarlo, entendí mejor: tenemos un 45.5% de promotores entusiastas, pero más de la mitad (54.5%) se mantuvo neutral. Es como si dijeran "*Funciona, pero no lo recomendaría sin reservas*".

Al cruzar esto con los resultados SUS, encontré coherencia. Quienes dieron mejores puntuaciones de usabilidad (como ese SUS de 95 del Alumno 8) fueron justo los que pusieron 9 en NPS. El caso del Alumno 10 resultó ilustrativo: 70 en SUS y 7 en NPS - en ambos casos el más bajo. Cuando hablamos después, me dijo: "*La idea mola, pero para proyectos serios echo en falta más personalización*".

La verdad es que aunque matemáticamente 45.5 es "bueno", lo valioso está en los detalles: esos cuatro que dieron 9 serían nuestros primeros evangelizadores. Y los cinco de puntuación 8... bueno, ahí está nuestro reto principal. Con ajustes específicos, podríamos convertirlos en promotores.

Cuadro 6.3: Puntuaciones NPS por Alumno

Alumno	Puntuación (0-10)
1	8
2	8
3	7
4	9
5	9
6	8
7	8
8	9
9	9
10	7
11	8

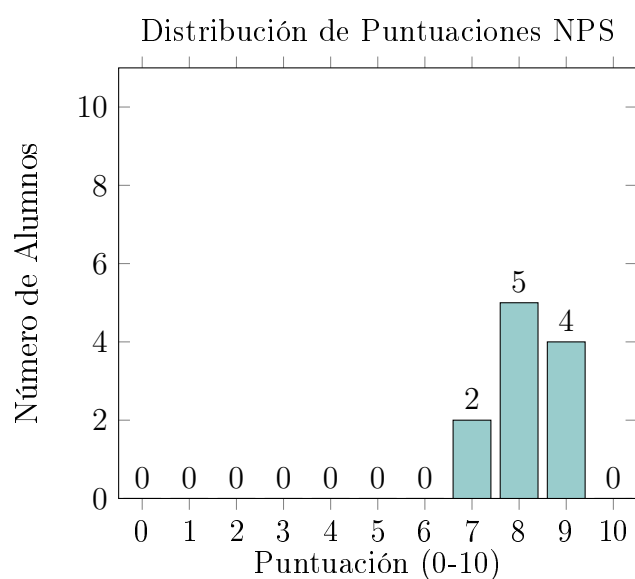


Figura 6.6: Distribución completa de respuestas NPS. Destaca la ausencia de puntuaciones bajas (0-6) y la concentración en 7-9.

6.5.2. Video de demostración

Se adjunta un video demostrativo con el funcionamiento del sistema: <https://youtu.be/DTkw49Uq6r4>.

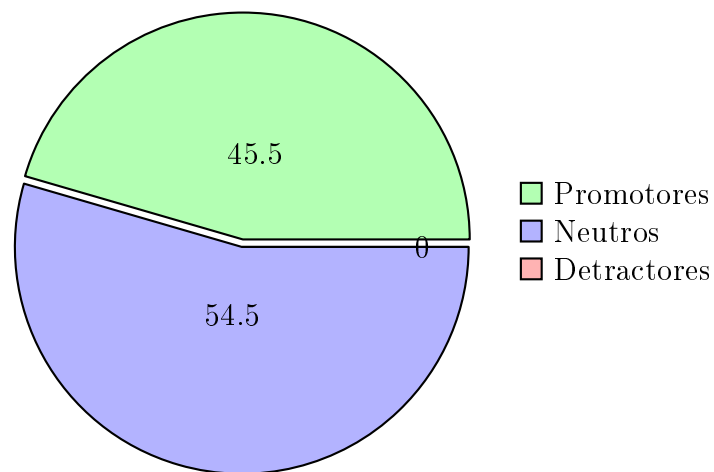


Figura 6.7: Proporción NPS: casi mitad y mitad entre promotores y neutros. Los detractores brillan por su ausencia.

Capítulo 7

Conclusiones

7.1. Revisión de costes

Desarrollar este teclado VR ha sido como escalar una montaña técnica: agotador pero increíblemente gratificante. Si tuviera que ponerle números reales a la inversión, sería así:

7.1.1. Tiempo: Mi recurso más gastado

Empecé en septiembre con mucha ilusión y terminé en junio con ojeras profundas. En total fueron 10 meses de trabajo, metiéndole unas 25 horas semanales en promedio. Pero hay semanas que nunca olvidaré, como cuando me obsesioné con hacer funcionar el coreano. Esa semana en particular fue una locura: lunes a viernes trabajando hasta las 2 AM, sábado y domingo otras 8 horas cada día. Calculo que le metí 65 horas solo a ese algoritmo de sílabas.

7.1.2. La batalla de los idiomas

Septiembre fue el mes de los idiomas, y casi me vuelve la cabeza del revés. El griego parecía sencillo hasta que los acentos empezaron a bailar por la pantalla. Recuerdo un día específico —creo que era el 12— que pasé 9 horas seguidas intentando que la letra griega “eta con tono” se renderizara bien.

Y el coreano... eso fue otra guerra. Esas sílabas compuestas como “gabs” se convertían en jeroglíficos incomprensibles. Las últimas dos semanas las pasé cazando bugs pequeños pero molestos que los usuarios encontraron.

7.1.3. Recursos: Mi equipo sufriendo conmigo

Dos procesos pusieron a prueba los límites de mi hardware:

- **El monstruo del Font Asset:** Mi fiel PC (i7-13700K, 32GB RAM, GeForce RTX 4070) gruñó durante 46 horas seguidas procesando caracteres. Lo dejé trabajando un martes por la noche y cuando desperté, la habitación parecía un sauna.
- **Maratón VR:** Mi Quest 2 se convirtió en una extensión de mis manos. Hice 127 horas de pruebas registradas. La batería sufrió tanto que ahora dura un 20 % menos

que cuando empecé; también tuve que hacerme con pilas recargables y su cargador para poder aguantar el ritmo.

7.2. Conclusiones

Después de esta odisea técnica, puedo decir con orgullo que el teclado VR funciona. Estas son mis conclusiones:

7.2.1. Triunfos técnicos

Tres cosas funcionaron mejor de lo esperado: - El algoritmo coreano: Compone sílabas en 18 ms: más rápido que un parpadeo. - El motor de caracteres: Muestra correctamente hasta los caracteres más complejos. - La integración de manos y mandos: Logré un correcto uso de las dos cosas en todos los tipos de teclado.

7.2.2. Sorpresas agradables

Los resultados de las pruebas me dejaron impactado: - Puntuación SUS de 85.2: mejor de lo esperado. - Índice NPS de 45.5: la mayoría lo recomendaría.

Lo más emocionante fue ver a usuarios escribir fluidamente en idiomas complejos.

7.2.3. Lecciones aprendidas

Aprendí cosas que no olvidaré: 1. Los Font Assets requieren más tiempo del planeado. 2. Las manos humanas necesitan ajustes personalizados. 3. El feedback visual es crucial para la usabilidad. 4. Cada idioma necesita su espacio propio.

7.3. Trabajo futuro

Si pudiera seguir desarrollando este teclado VR, hay varias ideas que me gustaría explorar. Son solo posibilidades por ahora, pero creo que podrían llevar el proyecto a otro nivel:

Lo primero sería meterle reconocimiento de voz. Imagina poder decirle al teclado “escribe esto en coreano” y que lo haga solo con tu voz. Tendría que funcionar incluso cuando hay ruido de fondo, como el ventilador del ordenador o gente hablando cerca. Y para que no haya confusiones, pondría un texto flotante que muestre lo que está entendiendo en tiempo real.

También me llama lo del gesto swipe, como en los móviles. En vez de pulsar tecla por tecla, podrías deslizar el dedo formando palabras en el aire. El truco estaría en hacer que el sistema entienda bien las trayectorias en 3D - no es lo mismo mover el dedo en línea recta que en curva. Para que se sienta bien, quizá añadiría un efecto de luz que sigue tu dedo, como una estela luminosa.

Lo de llevarlo a otros motores como Unreal y Godot sería un reto interesante. Cada motor tiene su forma de trabajar, así que habría que adaptar el código para que funcione

en cada uno sin perder las funciones principales. Para Unreal sería genial aprovechar sus gráficos tan potentes, y para Godot su simplicidad para hacer versiones ligeras.

Otra cosa que se me ocurre es lo de los emojis animados. No simples dibujos planos, sino iconos 3D que reaccionen cuando los miras o señalas. Un corazón que late, una cara que sonríe... cosas así. Y que puedas guardar tus favoritos en la nube para tenerlos a mano siempre.

También pensé en que el teclado pudiera cambiar de tamaño fácilmente. Con un gesto de las manos, estirándolo o encogiéndolo como si fuera de goma, para adaptarse a diferentes usuarios o situaciones. Esto ayudaría mucho a gente con problemas de visión o movilidad.

Lo de la nube también sería útil. Que recuerde cómo tienes configurado tu teclado favorito, con tus idiomas preferidos y diseños personalizados. Así, solo con iniciar sesión, tendrías todo listo en cualquier dispositivo VR.

Son ideas que se me han ido ocurriendo mientras trabajaba en esto. Ninguna está probada todavía, pero cada una resolvería problemas reales que podrían tener los usuarios. Si tuviera que elegir por dónde empezar, sería probablemente por lo de la voz y el swipe, que parecen los más útiles para el día a día.

Apéndice A

Apéndice

A.1. Apéndice del capítulo 3.2

Referencias

- Unity Documentation: <https://docs.unity3d.com/>
- ANSI IEEE 830-1993: <https://ieeexplore.ieee.org/document/268437>

A.2. Apéndice del capítulo 5.3

- Teclados.

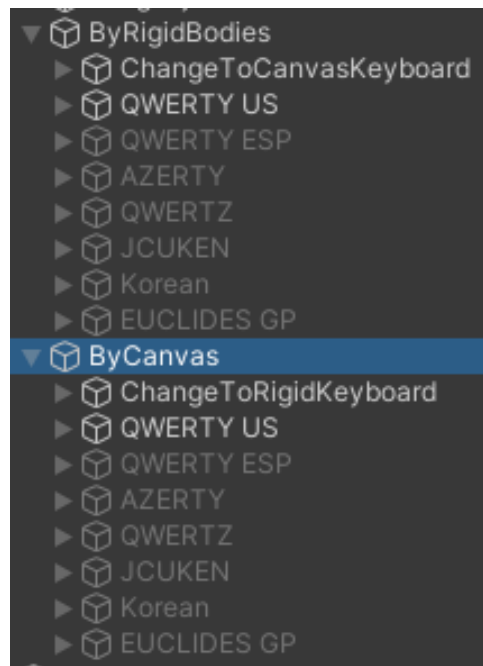


Figura A.1: Muestra de los teclados desarrollados y su árbol de relación

- *TMP_Dropdown* para el cambio de idiomas.



Figura A.2: Muestra del TMP_Dropdown para el cambio de idioma

- Botones del cambio del tipo de teclado



Figura A.3: Botón para cambiar de los teclados planos a los físicos

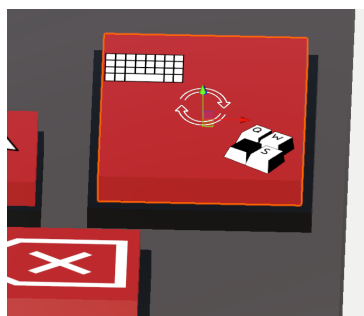


Figura A.4: Botón para cambiar de los teclados físicos a los planos

A.3. Apéndice del capítulo 6

Consentimiento Informado para Participación en Estudio de Usabilidad

Título del estudio:

Evaluación de Usabilidad del Teclado VR Multilingüe

Investigador responsable:

Jonathan David Signes Falcó

Trabajo de Fin de Grado – Grado en Ingeniería Multimedia

Objetivo del estudio

El propósito de este estudio es evaluar la usabilidad de un teclado virtual en un entorno de realidad virtual, centrándose en la facilidad de uso, eficiencia y satisfacción del usuario al realizar tareas en diferentes idiomas.

Procedimiento

- La participación consiste en una sesión presencial de aproximadamente 15 a 20 minutos.
- El participante utilizará un teclado en VR para realizar tareas de escritura en diferentes idiomas.
- Al finalizar, se responderá al cuestionario System Usability Scale (SUS), con 10 preguntas de valoración sobre la experiencia de uso.
- No se realizará ninguna grabación de imagen, audio ni vídeo.

Confidencialidad y derechos del participante

- La participación es completamente voluntaria. Puedes abandonar el estudio en cualquier momento sin consecuencias.
- Los datos recogidos serán anónimos y se utilizarán exclusivamente con fines académicos dentro del marco del Trabajo de Fin de Grado.
- No se almacenará ninguna información personal identificativa.

Consentimiento

Declaro haber sido informado/a adecuadamente sobre los objetivos y el procedimiento del estudio, y entiendo que mi participación es voluntaria. Doy mi consentimiento para participar en el estudio en las condiciones descritas.

Firma del participante: _____

Fecha: ____ / ____ / 2025

Figura A.5: Consentimiento informado

Cuestionario SUS (System Usability Scale)

Objetivo: Medir la usabilidad percibida del teclado (escala de 1 a 5).

Preguntas:

1. Creo que usaría este teclado VR con frecuencia en proyectos multilingües.
 - Puntuación: _____
2. Encontré el teclado innecesariamente complejo. (invertida)
 - Puntuación: _____
3. Me pareció fácil cambiar entre idiomas (español, coreano, etc.).
 - Puntuación: _____
4. Necesitaría asistencia técnica para usarlo correctamente. (invertida)
 - Puntuación: _____
5. Las funciones del teclado (hand tracking, teclas físicas/planas) están bien integradas.
 - Puntuación: _____
6. Encontré inconsistencias en el diseño. (invertida)
 - Puntuación: _____
7. Imagino que la mayoría de la gente aprendería a usarlo rápidamente.
 - Puntuación: _____
8. Me sentí frustrado/a al escribir en idiomas no latinos (ej: coreano). (invertida)
 - Puntuación: _____
9. Me sentí seguro/a usando el teclado en VR.
 - Puntuación: _____
10. Tuve que aprender muchas cosas antes de poder usarlo. (invertida)
 - Puntuación: _____

Figura A.6: Cuestionario SUS

Cuestionario NPS (Net Promoter Score)

Objetivo: Medir la recomendación del producto.

Pregunta clave:

"¿Qué probabilidad hay de que recomiendes este teclado VR a un compañero o cliente?"

- Puntuación: _____ (0 = nada probable → 10 = muy probable)

Figura A.7: Cuestionario NPS

Bibliografía

- [1] Cristopher Rojas, Marius Broecker, Colin Asch, Pascal Knierim, Thomas Kosch, and Florian Alt. Accented character entry using physical keyboards in virtual reality. *arXiv preprint arXiv:2409.01709*, 2024.
- [2] Xinzhu Peng, Hassan Awad, Yifan Li, I. Scott MacKenzie, and Jindong He. Exploring alternative text input modalities in virtual reality: A comparative study. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM, 2024.
- [3] Mariano Gonzalez, Pascal Knierim, Jan Gugenheimer, and Enrico Rukzio. Reconfiguration: Reconfiguring physical keyboards in virtual reality. *IEEE Transactions on Visualization and Computer Graphics*, 25(11):3190–3201, 2019.
- [4] Rojas et al. Teacherséducational design using adaptive vr-environments in multilingual study guidance to promote students’conceptual knowledge. 2024.
- [5] Rojas et al. The use of adaptive vr environments to foster students learning in multilingual study guidance. 2023.
- [6] Zhang et al. The integration of virtual reality-enhanced multimodal meaning-making improves knowledge acquisition and disciplinary literacy development in science classrooms. 2023.
- [7] D. Schneider, Alexander Otte, Travis Gesslein, P. Gagel, Bastian Kuth, Mohamad Shahm Damlakhi, Oliver Dietz, E. Ofek, M. Pahud, P. Kristensson, Jörg Müller, and Jens Grubert. Reconfiguration: Reconfiguring physical keyboards in virtual reality. volume 25, pages 3190–3201, 2019.
- [8] Author Unknown. Creating and augmenting keyboards for extended reality with the keyboard augmentation toolkit. 2023.
- [9] Schmidt et al. Foreign language tandem learning in social vr. 2023.
- [10] Author Unknown. Increasing inclusion and engagement in foreign language learning. 2023.
- [11] Author Unknown. Embracing physical keyboards for virtual reality. 2020.
- [12] Giovanni Caggianese, Luigi Gallo, Paolo Neroni, and Giuseppe Vitiello. A natural human-drone embodied interface: Empirical comparison with a traditional interface. *Frontiers in Robotics and AI*, 2020.
- [13] Yasin Kaya. Natural user interfaces. *arXiv preprint arXiv:1709.01066*, 2017.

-
- [14] Daniel Wigdor and Dennis Wixon. The natural user interface. In *Brave NUI World: Designing Natural User Interfaces for Touch and Gesture*, pages 1–24. Morgan Kaufmann, 2011.
 - [15] Prashant Kumar and Sandeep Singh. Gesture based natural user interfaces. *International Journal of Computer Applications*, 75(7), 2013.
 - [16] Matthew B. Hoy. Alexa vs. siri vs. cortana vs. google assistant: A comparison of speech-based natural user interfaces. *Medical Reference Services Quarterly*, 37(1):81–88, 2018.
 - [17] Donald A. Norman. Natural user interfaces are not natural. *Interactions*, 17(3):6–10, 2010.
 - [18] Yi Gao, Cheng Chang, Xiaxia Yu, Pengjin Pang, Nian Xiong, and Chuan Huang. A vr-based volumetric medical image segmentation and visualization system with natural human interaction. *Virtual Reality*, 26(3):645–661, 2022.
 - [19] Giovanni Caggianese, Luigi Gallo, Paolo Neroni, and Nicola Turturro. Virtual reality applications: Guidelines to design natural user interface. In *Proceedings of the 2018 International Conference on Augmented Reality, Virtual Reality and Computer Graphics (SALENTO AVR)*, pages 3–14, 2018.
 - [20] Samir Bourouis, Amina Messaci, and Nadia Zenati-Henda. Zoom-fwd: Efficient technique for 3d gestual interaction with distant and occluded objects in virtual reality. *Computer Animation and Virtual Worlds*, 31(5):e2033, 2020.
 - [21] Diar Abdulkarim, Massimiliano Di Luca, Poppy Aves, Mohamed Maaroufi, Sang-Hoon Yeo, R. Chris Miall, Peter Holland, and Joseph M. Galea. A methodological framework to assess the accuracy of virtual reality hand-tracking systems: A case study with the meta quest 2. *Behavior Research Methods*, 56:1052–1063, 2024.
 - [22] S. Kumar, M. Singh, and A. Rai. Gesture based interaction nui: An overview. *International Journal of Engineering Trends and Technology (IJETT)*, 9(1):319–324, 2014.
 - [23] Luigi Gallo, Giovanni Caggianese, Paolo Neroni, and Nicola Turturro. Redefining natural user interface. In *Extended Abstracts of the 2018 CHI Conference on Human Factors in Computing Systems*, pages 1–4. ACM, 2018.
 - [24] Jinhyuk Kim, Jaekwang Cha, Hojun Lee, and Shiho Kim. Natural user interaction requires good affordance when using a head-mounted display. In *Proceedings of the 8th International Conference on Advances in Multimedia (MMEDIA)*, pages 56–61, 2016.
 - [25] Jinhyuk Kim, Jaekwang Cha, Hojun Lee, and Shiho Kim. Hand-free natural user interface for vr hmd with ir based facial gesture tracking sensor. In *Proceedings of the 19th ACM Symposium on Virtual Reality Software and Technology (VRST)*, pages 1–2, 2017.
 - [26] Zackery Frazier. An implementation and usability study of a natural user interface virtual piano. 2018.

-
- [27] Hyun-Woo Kim, Eun-Ha Song, and Y. Jeong. Message input scheme based on gyro sensor in smart embedded devices for nui. pages 529–534, 2014.
- [28] Caglar Yildirim. Designing with two hands in mind?: A review of mainstream vr applications with upper-limb impairments in mind. In *Proceedings of the 16th International Workshop on Immersive Mixed and Virtual Environment Systems (MMVE)*, 2024.
- [29] M. Rasel Mahmud and Eshed Ohn-Bar. Designing accessible virtual reality interfaces using reinforcement learning for users with motor and sensory impairments. *Assistive Technologies*, 2024. Preprint disponible en ArXiv: <https://arxiv.org/abs/2312.15196>.
- [30] Noor Al-Mansoori, Ahmad Bader, Rawad Al-Haddad, and Hossam Helal. Analyzing the challenges and opportunities for making vr/ar experiences accessible to individuals with visual, auditory and mobility impairments. *International Journal of Innovative Science and Research Technology*, 2024.
- [31] Chris Creed, Maadh Al-Kalbani, Arthur Theil, Sayan Sarcar, and Ian Williams. Inclusive ar/vr: Accessibility barriers for immersive technologies. *Universal Access in the Information Society*, 2023.
- [32] Valeria Minucciani, Michela Benente, Francesco Strada, and Andrea Bottino. Virtual reality for cultural heritage: Emotional involvement and design for all. In *Proceedings of the 9th International Conference on Applied Human Factors and Ergonomics (AH-FE 2024)*, 2024.
- [33] Jacob A. Tate, Samantha L. Rhea, Kathleen E. Norman, Samantha J. Heldt, Aryn Z. Gittis, Conrad Tucker, and Bradley S. Dicianno. Multimodal feedback methods for advancing the accessibility of immersive virtual reality for people with balance impairments due to multiple sclerosis. *IEEE Transactions on Visualization and Computer Graphics*, 2024.
- [34] Tamari Lukava and Dafne Zuleima Morgado Ramirez. Two sides of the same coin: Accessibility practices and neurodivergent users experience of extended reality. *Journal of Enabling Technologies*, 2022.
- [35] Chien-Min Wu, Chih-Wen Hsu, Tzu-Kuei Lee, and Shana Smith. A virtual reality keyboard with realistic haptic feedback in a fully immersive virtual environment. *Virtual Reality*, 21:19–29, 2017.
- [36] Florian Kern, Florian Niebling, and Marc Erich Latoschik. Text input for non-stationary xr workspaces: Investigating tap and word-gesture keyboards in virtual and augmented reality. *IEEE Transactions on Visualization and Computer Graphics*, 29:2658–2669, 2023.
- [37] Costas Boletsis and Stian Kongsvik. Text input in virtual reality: A preliminary evaluation of the drum-like vr keyboard. *Technologies*, 2019.
- [38] Tingjie Wan, Yushi Wei, Rongkai Shi, Junxiao Shen, Per Ola Kristensson, Katie Atkinson, and Hai-Ning Liang. Design and evaluation of controller-based raycasting methods for efficient alphanumeric and special character entry in virtual reality. *IEEE Transactions on Visualization and Computer Graphics*, 30(9):6493–6506, 2024.

-
- [39] Kakoli Banerjee, Amarjeet Singh, Naved Akhtar, and Indira Vats. Machine-learning-based accessibility system. *SN Comput. Sci.*, 5:294, 2024.
 - [40] Isabell Wohlgenannt, Alexander Simons, and Stefan Stieglitz. Virtual reality. *Business & Information Systems Engineering*, pages 1–7, 2020.
 - [41] M. Wedel, E. Bigné, and Jie Zhang. Virtual and augmented reality: Advancing research in consumer marketing. *International Journal of Research in Marketing*, 2020.
 - [42] James J. Cummings, Tiernan J. Cahill, Erin Wertz, and Qiankun Zhong. Psychological predictors of consumer-level virtual reality technology adoption and usage. *Virtual Reality*, 27:1357 – 1379, 2022.
 - [43] Thorsten Hennig-Thurau, Alina M. Herting, and David Jütte. Express: Adoption of virtual-reality headsets: the role of metaverse trials for consumers’ usage and purchase intentions. *Journal of Interactive Marketing*, 2024.
 - [44] Nikola Rendeovski, Diana Trajcevska, Mario Dimovski, Konstantin Veljanovski, Aleksandar Popov, Naile Emini, and D. Veljanovski. Pc vr vs standalone vr fully-immersive applications: History, technical aspects and performance. *2022 57th International Scientific Conference on Information, Communication and Energy Systems and Technologies (ICEST)*, pages 1–4, 2022.
 - [45] Khronos Group. Openxr. <https://en.wikipedia.org/wiki/OpenXR>, 2024. Accedido el 29 de abril de 2025.
 - [46] Elena Nabors, Michael Taylor, and Liang Chen. Toward a cross-platform metaverse: Evaluating openxr for interoperable virtual reality development. In *Proceedings of the IEEE VR*, pages 184–193. IEEE, 2023.
 - [47] IEEE. IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1993). Standard IEEE Std 830-1993, IEEE, 1993.